

BASI DI DATI

Una **base di dati** è un insieme organizzato di dati, memorizzati in tabelle e utilizzati per il supporto allo svolgimento di un'attività.

È una raccolta di dati permanenti suddivisi in due categorie:

- I **metadati**, descritti da uno schema mediante il modello dei dati adottato dal DBMS e interrogabili con le stesse modalità previste per i dati. Descrivono fatti sullo schema dei dati, utenti autorizzati, applicazioni, parametri quantitativi sui dati, ecc.
- I **dati**, rappresentazioni di certi fatti conformi alle definizioni dello schema, con le seguenti caratteristiche:
 - Sono **organizzati in insiemi strutturati e omogenei**, tra i quali sono definite delle **relazioni**. La struttura dei dati e le relazioni sono descritte nello schema, utilizzando i meccanismi di astrazione del modello dei dati del DBMS.
 - Sono **molti**, in assoluto e rispetto ai metadati, e non possono essere gestiti in memoria temporanea.
 - Sono **accessibili mediante transazioni**, ovvero unità di lavoro atomiche che non possono avere effetti parziali.
 - Sono **protetti** sia dall'accesso da parte di utenti non autorizzati, sia da corruzione dovuta a malfunzionamenti hardware e software.
 - Sono **utilizzabili contemporaneamente** da utenti diversi.

Una base di dati deve essere **consistente** (rispettare i vincoli) e **omogenea** (contenere dati della stessa tipologia).

Le figure coinvolte nella costruzione di una base di dati sono:

- Il committente
- Il fornitore
- La manutenzione e messa a punto della base di dati/gestione del DBMS

Le basi di dati hanno un ruolo nel sistema informativo e nell'organizzazione di un'azienda.

Un **sistema informativo** di un'organizzazione è una combinazione di risorse, umane e materiali, e di procedure organizzate per la raccolta, l'archiviazione, l'elaborazione e lo scambio delle informazioni necessarie alle attività operative, di programmazione e controllo e di pianificazione strategica.

Un **sistema informativo automatizzato** è una parte del sistema informativo in cui le informazioni vengono raccolte, elaborate, archiviate e scambiate mediante un **sistema informatico**, ovvero l'insieme delle tecnologie informatiche e della comunicazione a supporto delle attività di un'organizzazione.

Nella costruzione di una base di dati è necessario evitare la creazione di **ridondanze**, ovvero la duplicazione di dati all'interno della stessa o di più tabelle.

Inoltre, all'interno di una tabella **non** devono essere presenti tuple (**righe**) **ripetute**.

Nei **sistemi informatici operativi**, i dati sono organizzati in basi di dati. Le applicazioni si usano per svolgere attività strutturate e ripetitive dell'azienda.

Le caratteristiche delle basi di dati sono garantite da un sistema per la gestione di basi di dati, il **DBMS**, che ha il controllo dei dati e li rende accessibili agli utenti autorizzati.

L'uso principale del DBMS è l'**OLTP (OnLine Transaction Processing)**, un approccio tradizionale all'elaborazione delle transazioni, utilizzato principalmente nei sistemi transazionali per eseguire operazioni quotidiane relativamente semplici che coinvolgono un numero limitato di dati dettagliati e che richiedono dati costantemente aggiornati.

Un **DBMS** è un sistema centralizzato o distribuito che offre opportuni linguaggi per definire lo schema di una base di dati, scegliere le strutture dati per la memorizzazione dei dati, memorizzare i dati rispettando i vincoli definiti nello schema, recuperare e modificare i dati interattivamente, mediante **query**, o da programmi.

Nei **sistemi informatici direzionali** i dati sono organizzati in **Data Warehouse (DW)** e gestiti da un sistema opportuno. Una Data Warehouse estrae i dati da una base di dati, li trasforma e li elabora.

L'uso principale delle DW è l'**OLAP (OnLine Analytical Processing)**, ovvero l'analisi dei dati di supporto alle decisioni aziendali, mediante operazioni analitiche complesse, non ripetitive, che possono coinvolgere grandi quantità di dati storici e aggregati, anche se non necessariamente i dati più attuali.

Differenze tra OLTP e OLAP:

	OLTP	OLAP
Scopi	Supporto operatività	Supporto decisioni
Utenti	Molti, esecutivi	Pochi, dirigenti e analisti
Dati	Analitici, relazionali	Sintetici, multidimensionali
Usi	Noti a priori	Poco prevedibili
Quantità di dati per attività	Bassa (decine)	Alta (milioni)
Orientamento	Applicazione	Soggetto
Aggiornamenti	Frequenti	Rari
Visione dei dati	Corrente	Storica
Ottimizzati per	Transazioni	Analisi dei dati

Il **modello relazionale** dei dati è il più diffuso tra i DBMS commerciali. Il meccanismo di astrazione fondamentale è la **relazione/tabella**, ovvero un insieme di righe con campi elementari. Lo schema di una relazione ne definisce il nome e descrive la struttura dei possibili elementi della relazione.

Esistono tre livelli d'astrazione del DBMS per la descrizione dei dati di una base di dati:

- Livello di **vista logica**: descrive come la struttura della base di dati deve apparire a una certa applicazione, aiutando nella protezione dei dati. Produce uno **schema esterno**.
(Ogni correntista può accedere solo alle informazioni relative al proprio conto corrente).
- Livello **logico**: descrive la struttura delle relazioni senza alcun riferimento alla loro organizzazione fisica in memoria. Produce uno **schema logico**.
(Corsi(IdeC CHAR(8), Titolo CHAR(20), Credito INT)).
- Livello **fisico**: descrive come organizzare fisicamente le relazioni in memoria e quali strutture dati utilizzare per facilitarne l'uso (la scelta della struttura dati impatta sulle prestazioni). Produce uno **schema interno**.
(Relazioni "Studenti" ed "Esami" organizzate in modo seriale, "Corsi" in modo sequenziale con indice).

Questa suddivisione in tre livelli del DBMS per la descrizione dei dati permette di garantire le proprietà di:

- **Indipendenza logica**: ogni livello è indipendente dall'altro; perciò, i programmi applicativi non devono essere modificati in seguito a modifiche del livello logico.
- **Indipendenza fisica**: i programmi applicativi non devono essere modificati in seguito a modifiche del livello fisico.

Un DBMS deve prevedere più modalità d'uso per soddisfare le esigenze delle diverse categorie d'utenti che possono accedere alla base di dati (interfaccia grafica, linguaggio di interrogazione e di programmazione). Esistono due tipi di linguaggi per l'interrogazione di una base di dati:

- **Data Manipulation Language (DML)**: per l'interrogazione e l'aggiornamento di istanze di una base di dati.
(SELECT Insegnamento FROM Orario WHERE Docente = "Mario Rossi").
- **Data Definition Language (DDL)**: per la definizione dei tre livelli (vista logica, logico e fisico) e altre operazioni generali.
(CREATE TABLE Orario (Insegnamento CHAR(20), Docente CHAR(20), Aula CHAR(4), Ora CHAR(5))).

Inoltre, un DBMS offre dei meccanismi per garantire le seguenti proprietà di una base di dati:

- **Integrità:** mantenimento delle proprietà specificate in fase di definizione delle relazioni (**vincoli d'integrità**).
- **Sicurezza:** protezione dei dati da usi non autorizzati.
- **Affidabilità:** protezione dei dati da malfunzionamenti e interferenze indesiderate dovute all'accesso concorrente ai dati da parte di più utenti.

Una **transazione** è una sequenza di azioni di lettura e scrittura, in memoria permanente, e di elaborazione di dati in memoria temporanea, avente le seguenti proprietà:

- **Atomicità:** le transazioni che terminano prematuramente vengono trattate come se non fossero mai accadute; perciò, i loro effetti sulla base di dati vengono annullati.
- **Persistenza:** le modifiche sulla base di dati di una transazione terminata normalmente sono permanenti; perciò, non sono alterabili da eventuali malfunzionamenti.
- **Serializzabilità:** nel caso di esecuzioni concorrenti di più transazioni, l'effetto complessivo è quello di un'esecuzione seriale.

La progettazione di una base di dati è l'attività più importante e consiste nel progettare la struttura dei dati e le sue applicazioni. Per progettare al meglio una base di dati è necessario che i dati siano un modello fedele del dominio del discorso.

Un **modello astratto** è la rappresentazione formale di fatti, idee e conoscenze relative a un fenomeno, mediante l'utilizzo di un linguaggio formale. Un modello è il risultato di un processo di interpretazione, guidato dalle idee e conoscenze possedute dal soggetto che le interpreta.

La progettazione di una base di dati si suddivide in quattro fasi, ciascuna centrata sulla modellazione:

- **Analisi dei requisiti:** definizione dei bisogni del committente in requisiti d'implementazione.
- **Progettazione concettuale:** traduzione dei requisiti in una struttura concettuale dei dati, dei vincoli e delle operazioni.
- **Progettazione logica:** traduzione dello schema concettuale in uno schema logico, espresso nel modello dei dati del sistema scelto per la realizzazione della base di dati.
- **Progettazione fisica:** produzione dello schema fisico.

Il **modello dei dati** è un insieme di **costrutti/regole** utilizzati per organizzare i dati di interesse e descriverne la dinamica. I **meccanismi di strutturazione/costruttori di tipo** sono i costrutti che consentono di definire la struttura dei dati o di definirne nuovi tipi. Il modello relazionale prevede il **costruttore relazione**, il quale permette di definire insiemi di righe omogenee.

Prima di affrontare la costruzione del modello di un universo del discorso è necessario porsi delle domande, come *“Che cosa si modella?”*, *“Con quali strumenti concettuali si modella?”*, *“Con quale linguaggio formale si definisce il modello?”* e *“Come si procede nella costruzione del modello?”*.

Le risposte caratterizzano i seguenti aspetti della modellazione:

- **Aspetto ontologico:** studio e modellazione di ciò che esiste. Si ha una conoscenza concreta dei fatti e una conoscenza astratta della struttura e dei vincoli sulla conoscenza concreta; inoltre, si ha una conoscenza procedurale sulle operazioni a cui può essere soggetta la conoscenza concreta (operazioni di base sugli utenti e come avverrà la comunicazione con il sistema informatico).
- **Aspetto logico:** il modello dei dati a oggetti. Il **modello dei dati** è un insieme di meccanismi di astrazione per descrivere la struttura della conoscenza concreta. Uno **schema** è la descrizione della struttura della conoscenza concreta e dei vincoli di integrità, usando un particolare modello di dati. Come notazione grafica utilizza una variante dei diagrammi ER (Entità-Relazione).
- **Aspetto linguistico**
- **Aspetto pragmatico**

La **conoscenza concreta** riguarda i fatti specifici che si intende rappresentare, ovvero:

- **Le entità con le loro proprietà:** le **entità** sono ciò che si desidera rappresentare come fatti o proprietà (un documento, un prestito, un utente). Le **proprietà** sono fatti che descrivono le caratteristiche di determinate entità (l'indirizzo di un utente). A seconda del contesto, i fatti possono essere interpretati come entità o proprietà, ma non c'è una regola fissa.
- **Le collezioni di entità omogenee:** una **proprietà** è una coppia (attributo, valore di un certo tipo). Ogni entità appartiene a un tipo che ne specifica la natura. Una **collezione** è un insieme, variabile nel tempo, di entità omogenee (ovvero dello stesso tipo).
- **Le associazioni tra entità:** un'istanza d'associazione è un fatto che correla due o più entità, stabilendo un legame logico tra di loro. Un'associazione $R(X, Y)$ tra due collezioni di entità X e Y è un insieme di istanze d'associazione tra elementi di X e Y , che varia nel tempo. **Il dominio dell'associazione è il prodotto cartesiano $X \times Y$.**

A ogni proprietà è associato un **dominio**, ovvero l'insieme dei possibili valori che può assumere. Una proprietà può essere:

- **Atomica:** il suo valore non è scomponibile; altrimenti, è **strutturata**.
- **Univoca:** il suo valore è unico; altrimenti, è **multivalore**.
- **Totale/obbligatoria:** ogni entità ha per essa un valore specificato; altrimenti, è **parziale/opzionale**.
- **Costante:** il suo valore non cambia; altrimenti, è **variabile**.
- **Calcolata:** il suo valore deve essere calcolato usando altre proprietà; altrimenti, è **non calcolata**.

Nella modellazione a oggetti, a ogni entità del dominio corrisponde un oggetto del modello.

Un **oggetto** è un'entità software con uno stato, un comportamento e un'identità, che modella un'entità dell'universo. Lo **stato** è modellato da un insieme di costanti o variabili con valori di qualsiasi complessità. Il **comportamento** è un insieme di procedure locali, chiamate **metodi**, che modellano le operazioni di base riguardo l'oggetto e le proprietà derivabili da altre. Un oggetto può rispondere a dei messaggi, restituendo dei valori memorizzati nello stato o calcolati con una procedura locale.

Una **classe** è un insieme di oggetti dello stesso tipo e modificabile con operatori per includere o estrarre elementi dall'insieme.

Il primo passo nella costruzione di un modello consiste nella classificazione delle entità del dominio con la definizione dei tipi degli oggetti che le rappresentano. Un **tipo oggetto** definisce l'insieme dei messaggi a cui può rispondere un insieme di possibili oggetti. I **nomi dei messaggi** sono anche detti **attributi degli oggetti**. I tipi oggetto non vengono rappresentati nei diagrammi, al contrario delle collezioni e delle associazioni. Però, la rappresentazione grafica di una collezione indica anche gli attributi del tipo oggetto associato.

Un'associazione $R(X, Y)$ è caratterizzata dalle proprietà strutturali di molteplicità e totalità e può avere quattro diverse cardinalità:

- **(1:N):** multivalore su X e univoca su Y .
- **(N:1):** univoca su X e multivalore su Y .
- **(N:M):** multivalore su X e multivalore su Y (molti a molti).
- **(1:1):** univoca su X e univoca su Y .

Per dare informazioni sulla cardinalità è utile ragionare in entrambe le direzioni delle classi (prima da C_1 a C_2 e poi da C_2 a C_1), fornendo un valore massimo e minimo per ciascuna delle due direzioni.

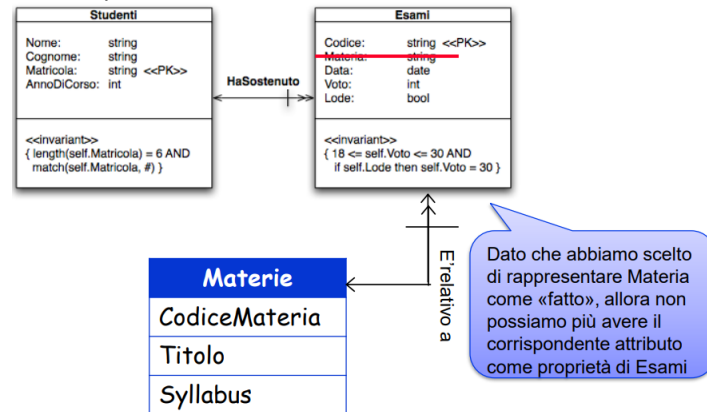
Un'associazione tra due collezioni C_1 e C_2 si rappresenta con una linea che collega le classi che rappresentano le due collezioni. La **linea** viene etichettata con il nome dell'associazione, che viene scelto utilizzando un predicato.

Un'associazione è **univoca (vincolo di univocità)** rispetto a X se, per ogni elemento x di X , esiste al massimo un elemento y di Y associato a x ; se questo vincolo non vale, l'associazione è **multivalore** rispetto a X . L'univocità si rappresenta con una singola freccia $\longrightarrow$ mentre, l'essere multivalore con una doppia freccia $\longrightarrow\!\!\longrightarrow$.

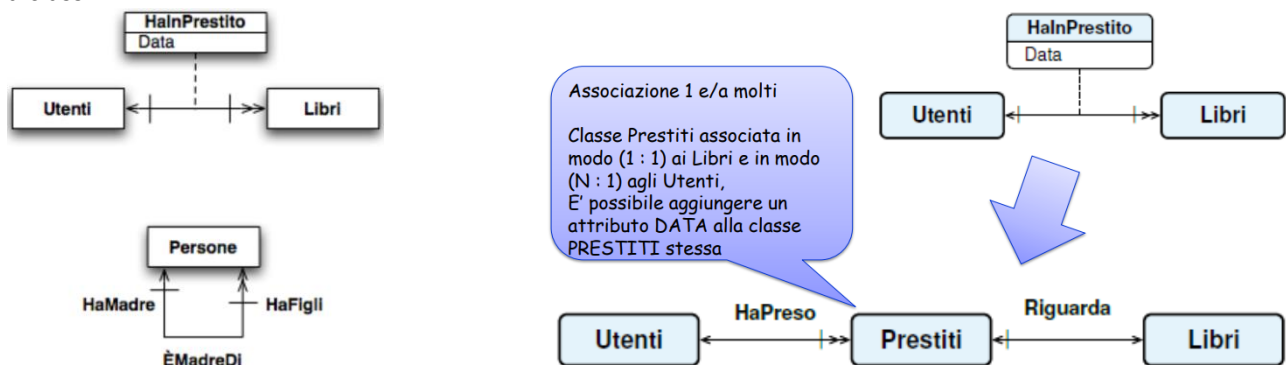
Un'associazione è **totale (vincolo di totalità)** su X, se per ogni elemento x di X, esiste almeno un elemento di y di Y che è associato a x; se questo vincolo non vale, l'associazione è **parziale** rispetto a X. La parzialità si rappresenta con un taglio sulla linea, vicino alla freccia, ---| mentre la totalità non presenta tale taglio.

(---|) si legge come "o nessuno o molti", mentre --- si legge come "uno solo".

Se durante la progettazione concettuale di una base di dati si decide di rappresentare la proprietà di una classe come fatto, allora il corrispondente attributo dev'essere rimosso dalla classe in cui era presente.



Nel modello a oggetti, le associazioni vengono modellate con un costrutto apposito, il quale le permette di avere delle proprietà e di essere ricorsive. Inoltre, è possibile "promuovere" delle associazioni con proprietà a classi.



Le associazioni ricorsive sono relazioni binarie tra gli elementi di una stessa classe. È necessario etichettare la linea con il nome dell'associazione e dei nomi che specifichino il ruolo che hanno le componenti in un'istanza dell'associazione.

Le classi di entità sono organizzate in una **gerarchia di specializzazione/generalizzazione**, la quale può essere semplice o multipla. Una **sottoclasse** è una classe della gerarchia minore di altre; una **superclasse** è una classe della gerarchia maggiore di altre. In una gerarchia, gli elementi di una sottoclasse sono un sottoinsieme degli elementi della superclasse. Inoltre, una gerarchia consente l'ereditarietà delle proprietà di una superclasse da parte di una sottoclasse.

(La classe "Persone" è una generalizzazione delle classi "(Non) Patentati", "(Non) Studenti").

Tra i tipi oggetto è presente una relazione di sottotipo, avente le seguenti proprietà:

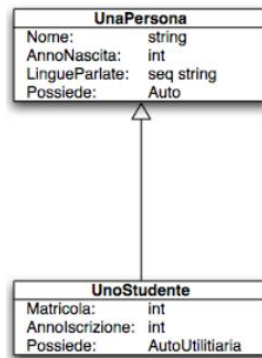
- **Asimmetricità** (se A è in relazione con B, allora B non è in relazione con A).
- **Riflessività** (A è in relazione con sé stesso).
- **Transitività** (se A è in relazione con B e B è in relazione con C, allora A è in relazione con C).
- **Sostitutività** (se A è sottotipo di B, allora gli elementi di A possono essere usati in ogni contesto in cui possono apparire i valori di tipo B). In particolare:
 - Gli elementi di A hanno tutte le proprietà degli elementi di B.

- Per ogni proprietà p in B , il suo tipo in A è un sottotipo del suo tipo in B .

L'**ereditarietà** permette di:

- Definire un tipo oggetto a partire da un altro (definizione di sottotipi).
- Implementare un tipo oggetto a partire da un'altra implementazione (definizione implementazione di sottotipi).

In questo caso, gli attributi possono solo essere aggiunti e possono essere ridefiniti solo specializzandone il tipo.

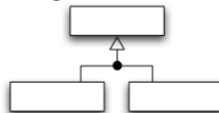


Un **sottoinsieme** è una relazione di sottoclasse definita tra le classi, che presenta le seguenti proprietà:

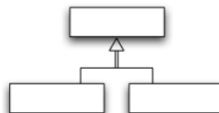
- **Asimmetricità, riflessività e transitività.**
- **Vincolo intensionale** (se A è sottoclasse di B , allora il tipo degli elementi di A è sottotipo del tipo degli elementi di B).
- **Vincolo estensionale** (se A è sottoclasse di B , allora gli elementi di A sono un sottoinsieme degli elementi di B).

Su un insieme di sottoclassi si possono avere due vincoli:

- **Disgiunzione:** ogni coppia di sottoclassi è disgiunta, ovvero priva di elementi comuni.

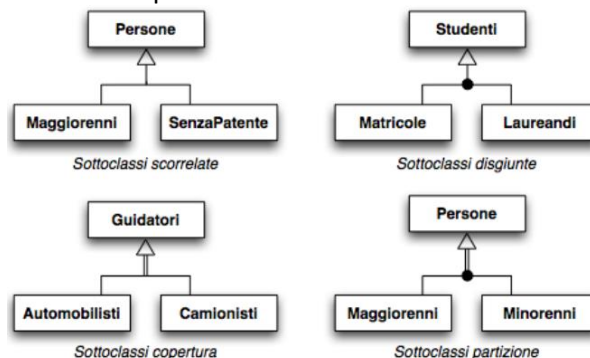


- **Copertura:** l'unione degli elementi delle sottoclassi coincide con tutti gli elementi della superclasse.



Se su una coppia di sottoclassi sono attivi entrambi i vincoli, si dicono **sottoclassi partizione**.

Perciò, le relazioni tra sottoinsiemi che si possono avere sono:



Un tipo può essere definito per ereditarietà a partire da un unico supertipo (**ereditarietà singola**) o da più supertipi (**ereditarietà multipla**).

L'**ereditarietà multipla** permette a una sottoclasse di ereditare attributi e metodi da più supertipi. Un problema si può verificare se uno stesso attributo viene ereditato da più supertipi diversi.

(Se due supertipi diversi forniscono un attributo con lo stesso nome ma comportamenti diversi, il sottotipo può avere difficoltà a decidere quale implementazione utilizzare).

La **conoscenza astratta** descrive la struttura e le restrizioni sui valori possibili della conoscenza concreta e sui modi in cui essi possono evolvere nel tempo (vincoli d'integrità statici e dinamici); inoltre, descrive le regole per derivare nuovi fatti da altri noti.

I **vincoli** possono essere descritti in modo **dichiarativo** con formule del calcolo dei predicati (modo preferito), oppure mediante **controlli** da eseguire nelle operazioni di base o degli utenti.

Nel dettaglio, la progettazione di una base di dati si suddivide in:

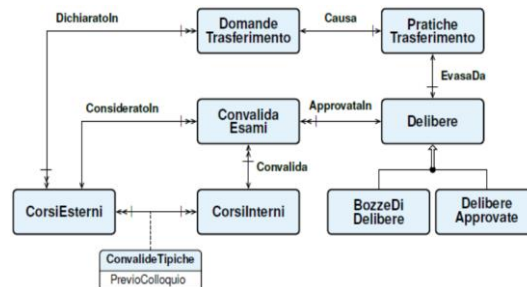
- **Analisi dei requisiti:** si raccoglie, organizza e strutturano i requisiti informali in modo da eliminare ambiguità e imprecisioni. Si costruisce un glossario dei termini e uno schema di settore.

Termine	Descrizione	Sinonimi	Collegamenti
Partecipante	Persona che partecipa ai corsi	Studiante	Corso, Società
Docente	Docente dei corsi. Può essere esterno	Insegnante	Corso
Corso	Corso organizzato dalla società. Può avere più edizioni.	Seminario	Docente
Datori	Ente presso cui i partecipanti lavorano o hanno lavorato	Posti	Partecipante

Dopo aver definito i dati è necessario stabilire le specifiche sulle operazioni da effettuare. È importante utilizzare la stessa terminologia usata per i dati e indicare la frequenza con cui vengono effettuate certe operazioni.

(Ad esempio, **OPERAZIONE 1:** Assegna un partecipante a un'edizione di corso (50 volte al giorno)).

- **Progettazione concettuale:** si identificano le classi, le associazioni, gli attributi, le chiavi, le sottoclassi e le generalizzazioni. Si costruisce uno schema concettuale usando il diagramma ER.



- **Progettazione logica:** si trasforma lo schema concettuale in uno schema logico adatto a un modello di dati specifico, come il modello relazionale o il modello a oggetti. Si applicano delle regole di trasformazione e si definiscono le viste esterne.
- **Progettazione fisica:** si ottimizzano le prestazioni della base di dati, tenendo conto delle caratteristiche del sistema fisico e delle operazioni più frequenti. Si scelgono le strutture di memorizzazione, gli indici e le tecniche di accesso ai dati.
- **Realizzazione:** si implementa la base di dati usando un DBMS e uno o più linguaggi di interrogazione e manipolazione dei dati, come SQL.

Il **modello relazionale** si basa sul concetto matematico di **relazione**, con una variante. Le relazioni vengono rappresentate mediante **tabelle** con righe distinte e colonne omogenee; perciò:

- Il **prodotto cartesiano** $D_1 \times \dots \times D_n$ è l'insieme di tutte le tuple $(d_1, \dots, d_n) : d_1 \in D_1, \dots, d_n \in D_n$

$$D_1 = \{a, b\}$$

$$D_2 = \{x, y, z\}$$

$$\text{prodotto cartesiano } D_1 \times D_2$$

a	x
a	y
a	z
b	x
b	y
b	z

- Una **relazione matematica** su $D_1, \dots, D_n$ è un sottoinsieme di $D_1 \times \dots \times D_n$

$$\text{una relazione } r \subseteq D_1 \times D_2$$

a	x
a	z
b	y

Dove $D_1, \dots, D_n$ sono **n insiemi**, anche non distinti, e i **domini** della relazione.

Una relazione è un **insieme**, quindi:

- **Non c'è ordinamento** tra le tuple.
- Le tuple sono **distinte**.
- Ciascuna tupla è **ordinata**, ovvero l' i -esimo valore proviene dall' i -esimo dominio.

Ciascun dominio D_i ha **ruoli** diversi, distinguibili attraverso la posizione (la struttura è **posizionale**); perciò, a ciascun dominio si associa un nome/**attributo** che ne descrive il ruolo.

Una **tupla** su un insieme di attributi X è una funzione t che associa a ciascun attributo A un valore del suo dominio. Una **relazione su X** è un insieme di tuple su X .

Una tabella rappresenta una relazione se:

- I valori di ogni colonna sono tra loro **omogenei**.
- Le righe sono **diverse** tra loro.
- Le intestazioni delle colonne sono **diverse** tra loro.

Inoltre, l'ordinamento tra le righe o le colonne è **irrilevante**.

Il modello relazionale si basa su valori; perciò, i riferimenti tra dati e relazioni diverse sono rappresentati tramite valori dei domini che compaiono nelle tuple.

In ogni base di dati si distinguono:

- Lo **schema** (invariante nel tempo), che descrive la struttura della base di dati (aspetto intensionale, le intestazioni delle tabelle)
- L'**istanza** (variante nel tempo), ovvero i valori attuali (aspetto estensionale, il corpo di ogni tabella).

Studenti	Nome	Matricola	Provincia	AnnoNascita	Schema di relazione Istanza di Relazione o estensione della relazione
	Isaia	071523	PI	1982	
	Rossi	067459	LU	1984	
	Bianchi	079856	LI	1983	
	Bonini	075649	PI	1984	

I meccanismi per definire una base di dati con il modello relazionale sono la **tupla** e la **relazione**:

- Un **tipo tupla T** è un insieme finito di coppie (*Attributo, Tipo elementare*)
- Lo **schema della relazione R** si indica con $R(T)$, dove T è un tipo tupla.
- Lo **schema di una base di dati** è un insieme di schemi di relazione $R_i(T_i)$
- Un'**istanza** di uno schema $R(T)$ è un insieme finito di tuple di tipo T

Uguaglianza di due tipi tupla/due tuple/due tipi relazione sono tutti sinonimi.

Una struttura basata su valori permette di:

- Avere un'**indipendenza dalle strutture fisiche**, le quali possono cambiare dinamicamente. Questo perché, la rappresentazione logica dei dati (costituita solo dai valori), non fa riferimento a quella fisica.
- Rappresentare solo ciò che è rilevante dal punto di vista dell'applicazione.
- Portare più facilmente i dati da un sistema all'altro.
- Avere **puntatori direzionali**.

Uno **schema di relazione $R(A_1, \dots, A_n)$** è un nome R con un insieme di attributi $A_1, \dots, A_n$

Uno **schema di base di dati $R = \{R_1(X_1), \dots, R_k(X_k)\}$** è un insieme di schemi di relazione dove $X_1, \dots, X_k$ sono insiemi di attributi.

Una **tupla t** su un insieme di attributi T è una funzione che associa a ciascun attributo $A \in T$ un valore del dominio di A , dove:

- $t[A]$ denota il valore della tupla t sull'attributo $A \in T$
- $t[X]$ denota i valori della tupla t sugli attributi $X \in T$

Una **relazione/istanza di relazione** su uno schema $R(X)$ è un insieme r di tuple su X

Un'istanza di **base di dati** su uno schema $R = \{R_1(X_1), \dots, R_k(X_k)\}$ è l'insieme delle relazioni $r = \{r_1, \dots, r_n\}$, con r_i relazione su R_i

Il modello relazionale impone ai dati una struttura rigida. Le informazioni sono rappresentate per mezzo di tuple e solo alcuni formati di tuple sono ammessi, ovvero quelli che corrispondono agli schemi di relazione.

Il **valore nullo NULL** denota l'assenza di un valore del dominio (esso non è un valore del dominio); perciò, $t[A]$, per ogni attributo A , è un valore del dominio $dom(A)$ oppure il valore nullo **NULL** (è necessario imporre restrizioni sulla presenza di valori nulli).

Uno **schema relazionale** è costituito da un insieme di schemi di relazione e da un insieme di vincoli d'integrità sui possibili valori delle estensioni delle relazioni.

Un **vincolo d'integrità** è una proprietà che dev'essere soddisfatta dalle istanze che rappresentano informazioni corrette per l'applicazione. Si esprime mediante una funzione booleana (un **predicato**); perciò, associa a ogni istanza il valore vero o falso.

I vincoli d'integrità:

- Permettono di avere una descrizione più accurata della realtà.
- Forniscono un contributo alla qualità dei dati.
- Sono utili in fase di progettazione.
- Vengono utilizzati dai DBMS nell'esecuzione delle query.

Alcuni tipi di vincoli sono:

- **Vincoli intrarelazionali**: devono essere rispettati dai valori contenuti nella relazione considerata.
Si dividono in:
 - **Vincoli su valori/di dominio**: coinvolgono un solo attributo.
(Ad esempio, $(Voto \geq 18) \text{ AND } (Voto \leq 30)$)
 - **Vincoli di tupla**: esprimono condizioni sui valori di ciascuna tupla, indipendentemente dalle altre tuple.
- **Vincoli interrelazionali**: devono essere rispettati dai valori contenuti in relazioni diverse.

Una **chiave** è un insieme di attributi che identificano le tuple di una relazione.

Un insieme K di attributi è **superchiave** per r se r non contiene due tuple distinte t_1 e t_2 , con $t_1[K] = t_2[K]$; inoltre, K è **chiave** per r se è una superchiave minimale per r , ovvero non contiene un'altra superchiave.

Matricola	Cognome	Nome	Corso	Nascita
27655	Rossi	Mario	Ing Inf	5/12/78
78763	Rossi	Mario	Ing Inf	3/11/76
65432	Neri	Piero	Ing Mecc	10/7/79
87654	Neri	Mario	Ing Inf	3/11/76
67653	Rossi	Piero	Ing Mecc	5/12/78

Ad esempio, *Matricola* è una chiave, dato che è una superchiave ed è minimale (in questo caso contiene un solo attributo).

Perciò, i vincoli corrispondono a proprietà del mondo reale modellato dalla base di dati. Interessano a livello di schema, con riferimento a tutte le istanze possibili. A uno schema viene associato un insieme di vincoli e vengono considerate **corrette/valide/ammissibili** le istanze che soddisfano tutti i vincoli. Può capitare che, per caso, un'istanza soddisfi altri vincoli.

Una relazione non può contenere tuple distinte, ma con valori uguali, dato che una relazione è un sottoinsieme del prodotto cartesiano.

Ogni relazione ha sicuramente come superchiave l'insieme di **tutti gli attributi su cui è definita**; perciò, ogni relazione ha **almeno una chiave**.

L'esistenza delle chiavi garantisce l'accessibilità a ciascun dato della base di dati; inoltre, le chiavi permettono di correlare i dati in relazioni diverse, dato che il modello relazionale è basato su valori.

La presenza di valori nulli tra i valori di una chiave non permette l'identificazione delle tuple e la realizzazione di riferimenti ad altre relazioni.

Una **chiave primaria** è una chiave su cui **non** sono ammessi valori nulli. L'attributo sottolineato rappresenta la chiave primaria.

<u>Matricola</u>	Cognome	Nome	Corso	Nascita
86765	NULL	Mario	Ing Inf	5/12/78
78763	Rossi	Mario	Ing Civile	3/11/76
65432	Neri	Piero	Ing Mecc	10/7/79
87654	Neri	Mario	Ing Inf	NULL
43289	Neri	Mario	NULL	5/12/78

Nel modello relazionale, le informazioni in relazioni diverse sono correlate, mediante correlazioni coerenti, attraverso valori comuni. In particolare, vengono spesso presi in considerazione i valori delle chiavi primarie.

Un **vincolo di integrità referenziale (chiave esterna)** tra gli attributi X di una relazione R_1 e un'altra relazione R_2 impone ai valori su X in R_1 di comparire come valori della chiave primaria di R_2 . L'attributo con l'asterisco rappresenta la chiave esterna.

Studenti	Nome	<u>Matricola</u>	Provincia	AnnoNascita
	Isaia	071523	PI	1982
	Rossi	067459	LU	1984
	Bianchi	079856	LI	1983
	Bonini	075649	PI	1984

Esami	Materia	<u>Candidato*</u>	Data	Voto
	BD	071523	12/01/06	28
	BD	067459	15/09/06	30
	FP	079856	25/10/06	30
	BD	075649	27/06/06	25
	LMM	071523	10/10/06	18

Chiave esterna ↑ Vincolo di integrità referenziale

In questo caso, il vincolo d'integrità referenziale è tra l'attributo *Candidato* della relazione *Esami* e la relazione *Studenti*.

Nel caso in cui dovesse venire eliminata una tupla dalla tabella riferita, causando una violazione, è possibile:

- Rifiutare l'operazione.
- Effettuare un'eliminazione in cascata.
- Introdurre valori nulli.

L'obiettivo della **progettazione logica** è quello di “tradurre” lo schema concettuale in uno **schema logico relazionale** che rappresenti gli stessi dati in maniera corretta ed efficiente. Non si tratta di una vera traduzione, dato che alcuni costrutti dello schema concettuale non sono direttamente rappresentabili e, nel modello logico, è necessario tenere conto delle prestazioni. Questo richiede una ristrutturazione del modello concettuale.

La progettazione logica richiede i seguenti dati:

- In ingresso:
 - Uno schema concettuale.
 - Informazioni sul carico applicativo (dimensione dei dati e caratteristiche delle operazioni).
 - Un modello logico.
- In uscita:
 - Uno schema logico.
 - Una documentazione associata.

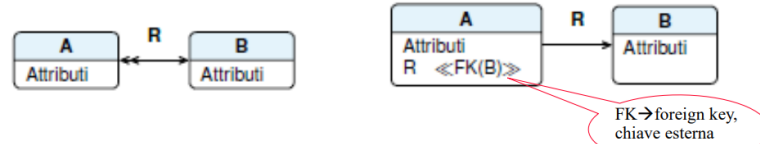
La trasformazione di uno schema a oggetti in uno schema relazionale avviene mediante i seguenti passi:

1. Rappresentazione delle associazioni 1:1 e 1:N (uno a molti).

Per rappresentare le associazioni 1:1 si aggiunge la **chiave esterna** a una qualunque delle due relazioni che riferisce l'altra, preferendo, in presenza di un vincolo di totalità, quella avente l'associazione totale.



Per rappresentare le associazioni 1:N (uno a molti) si aggiunge, agli attributi della relazione la cui associazione è univoca, una **chiave esterna** che riferisce l'altra relazione.

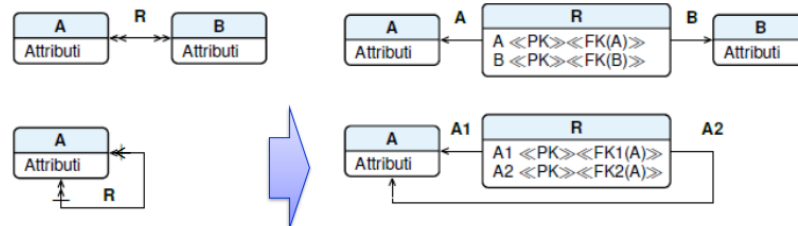


La direzione dell'associazione rappresentata dalla chiave esterna è detta **"la diretta"** dell'associazione. In questo caso, i vincoli sulla cardinalità delle associazioni sono:

- Univocità della diretta.
- Totalità della diretta: rappresentata da un vincolo *NOT NULL* sulla chiave esterna.
- Univocità dell'inversa e totalità della diretta: rappresentata da un vincolo *NOT NULL* e un vincolo di chiave sulla chiave esterna.

2. Rappresentazione delle associazioni N:M o non binarie.

Per rappresentare le associazioni N:M (molti a molti) si aggiunge allo schema una nuova relazione che contiene **due chiavi esterne** che riferiscono le due relazioni coinvolte. La **chiave primaria** di questa relazione è costituita dall'insieme di tutti i suoi attributi, ovvero le due chiavi esterne.



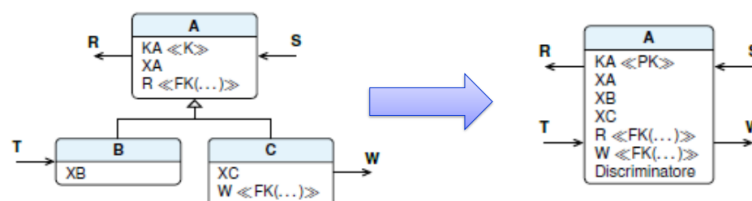
3. Rappresentazione delle gerarchie di inclusione.

Dato che il modello relazionale non può rappresentare direttamente le gerarchie, è necessario eliminarle e sostituirle con classi e relazioni, mediante una delle seguenti tecniche:

1. Relazione unica: accorpamento delle figlie della gerarchia nel genitore.

Se A_0 è la classe genitore di A_1 e A_2 , le classi A_1 e A_2 vengono **eliminate** e **accorpate** ad A_0 , alla quale viene aggiunto un **attributo discriminatore** che indica da quale delle classi figlie deriva una certa istanza; gli attributi di A_1 e A_2 vengono assorbiti dalla classe genitore e assumono valore nullo sulle istanze provenienti dall'altra classe.

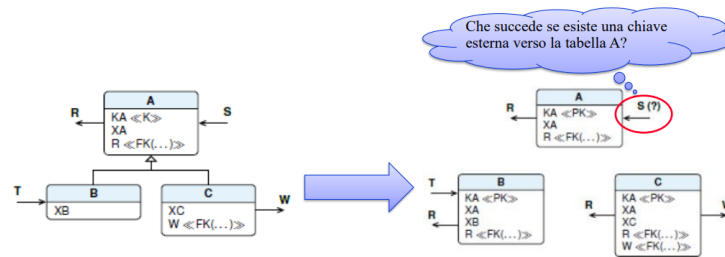
Infine, una relazione relativa a **solo una** delle classi figlie viene acquisita dalla classe genitore e avrà comunque **cardinalità minima uguale a zero**, dato che gli elementi dell'altra classe non contribuiscono alla relazione.



2. Partizionamento orizzontale: accorpamento del genitore della gerarchia nelle figlie.

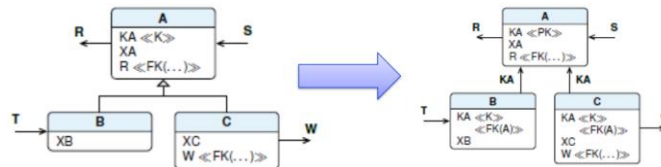
La classe genitore A_0 viene **eliminata** e le classi figlie A_1 e A_2 **ereditano** le proprietà della classe genitore, ovvero l'identificatore, gli attributi e le relazioni.

Dato che il partizionamento orizzontale divide gli elementi della superclasse in più relazioni diverse, non è possibile mantenere un vincolo referenziale **verso la superclasse estesa**; perciò, questa tecnica non viene utilizzata se nello schema relazionale grafico è presente una freccia che entra nella superclasse.

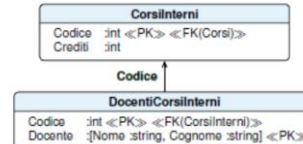
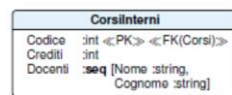


3. Partizionamento verticale: sostituzione della gerarchia con relazioni.

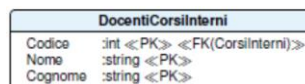
La gerarchia si trasforma in **due associazioni 1:1** che legano rispettivamente la classe genitore con le classi figlie. In questo caso non avviene alcun trasferimento di attributi o di associazioni e le classi figlie A_1 e A_2 sono identificate esternamente dalla classe genitore A_0 . Nello schema ottenuto è necessario **aggiungere dei vincoli**; ogni occorrenza di A_0 non può partecipare **contemporaneamente** alle due associazioni e, se la gerarchia è **totale**, deve partecipare ad **almeno una** delle due.



4. Identificazione delle chiavi primarie.
5. Rappresentazione degli attributi multivalore.



6. Appiattimento degli attributi composti.



L'obiettivo è quindi quello di **rappresentare le stesse informazioni, minimizzare la ridondanza** e produrre uno **schema comprensibile** per facilitare la scrittura e la manutenzione delle applicazioni.

Nelle basi di dati, per effettuare operazioni sullo schema si utilizza il **Data Definition Language (DDL)**, mentre per effettuare operazioni sui dati si utilizza il **Data Manipulation Language (DML)**, il quale si divide in **Data Query Language (DQL)** per effettuare query, e **aggiornamento dati** per inserire, cancellare o modificare i dati.

L'**algebra relazionale** è un insieme di operatori su relazioni che danno come risultato delle relazioni; non si utilizza come linguaggio di interrogazione dei DBMS, ma come rappresentazione interna delle interrogazioni. Il **calcolo relazionale** è un linguaggio dichiarativo di tipo logico dal quale è stato derivato l'SQL.

Gli operatori dell'algebra relazionale sono:

- **Unione $R \cup S$** : unisce tutte le tuple delle relazioni in una sola (i duplicati vengono uniti).

Laureati			Quadri		
Matricola	Nome	Età	Matricola	Nome	Età
7274	Rossi	42	9297	Neri	33
7432	Neri	54	7432	Neri	54
9824	Verdi	45	9824	Verdi	45

Laureati $\cup$ Quadri		
Matricola	Nome	Età
7274	Rossi	42
7432	Neri	54
9824	Verdi	45
9297	Neri	33



- **Intersezione $R \cap S$:** crea una relazione avente le tuple in comune delle relazioni coinvolte



Matricola	Nome	Età
7274	Rossi	42
7432	Neri	54
9824	Verdi	45

Matricola	Nome	Età
9297	Neri	33
7432	Neri	54
9824	Verdi	45

Matricola	Nome	Età
7432	Neri	54
9824	Verdi	45

- **Differenza $R - S$:** crea una relazione non avente le tuple in comune delle relazioni coinvolte.

Matricola	Nome	Età
7274	Rossi	42
7432	Neri	54
9824	Verdi	45

Matricola	Nome	Età
9297	Neri	33
7432	Neri	54
9824	Verdi	45

Matricola	Nome	Età
7274	Rossi	42



- **Ridenominazione:** modifica lo schema, momentaneamente, lasciando inalterata l'istanza dell'operando. Si effettua mediante l'operatore **monadico ρ** avente un solo argomento.

Madre	Figlio
Eva	Abele
Eva	Set
Sara	Isacco

Genitore	Figlio
Eva	Abele
Eva	Set
Sara	Isacco

- **Selezione:** produce un risultato che ha lo stesso schema dell'operando e contiene un sottoinsieme delle tuple dell'operando. Si effettua mediante l'operando **monadico σ** , con la condizione a pedice.

Matricola	Cognome	Filiale	Stipendio
7309	Rossi	Roma	55
5998	Neri	Milano	64
5698	Neri	Napoli	64

$\sigma_{\text{Stipendio} > 50}(\text{Impiegati})$

Nella condizione è possibile utilizzare i connettivi logici $\wedge$ (AND), $\vee$ (OR), $\neg$ (NOT). Inoltre, la condizione è vera solo per valori non nulli (per far riferimento a valori non nulli si utilizzano le condizioni *IS NULL* oppure *IS NOT NULL*).

- **Proiezione:** produce un risultato che ha parte degli attributi dell'operando e contiene tuple che contribuiscono a tutte le tuple dell'operando, ristrette agli attributi nella lista. Si effettua mediante l'operando **monadico π** e **non presenta la proprietà distributiva** ($\pi_A(R_1 - R_2) \neq \pi_A(R_1) - \pi_A(R_2)$).

Matricola	Cognome
7309	Neri
5998	Neri
9553	Rossi
5698	Rossi

$\pi_{\text{Matricola, Cognome}}(\text{Impiegati})$

Una proiezione contiene al massimo tante tuple quante l'operando e:

- Se X è una superchiave di R , allora $\pi_X(R)$ contiene esattamente tante tuple quante R
- Se X non è una superchiave, potrebbero esistere valori ripetuti su quegli attributi ma che vengono rappresentati una sola volta.
- **Join:** permette di correlare dati in relazioni diverse. Esistono diversi tipi di join:
 - **Join naturale:** operatore binario generalizzabile che correla dati in relazioni diverse, sulla base di valori uguali in attributi con lo stesso nome.

Produce un risultato sull'unione degli attributi degli operandi, con tuple ottenuto combinando le tuple degli operandi con valori uguali sugli attributi in comune.

Siano $R_1(X_1)$ e $R_2(X_2)$, allora $R_1 \bowtie R_2$ è una relazione su $X_1 \cup X_2$, dove:

$$R_1 \bowtie R_2 = \{t \in X_1 \cup X_2 \mid \exists t_1 \in R_1 \wedge t_2 \in R_2, \text{ con } t[X_1] = t_1 \wedge t[X_2] = t_2\}$$

Impiegato	Reparto	Reparto	Capo
Rossi	A	A	Mori
Neri	B	B	Bruni
Bianchi	B		

Impiegato	Reparto	Capo
Rossi	A	B
Neri	B	B
Bianchi	B	B

Impiegato	Reparto	Capo
Rossi	A	D
Neri	B	C
Bianchi	B	

- Il join di R_1 e R_2 contiene un numero di tuple tra $0 \leq |R_1 \bowtie R_2| \leq |R_1| \times |R_2|$
- Se il join tra R_1 e R_2 è completo, allora contiene un numero di tuple almeno uguale al massimo tra $|R_1|$ e $|R_2|$
- Se il join coinvolge una chiave B di R_2 , il numero di tuple è tra $0 \leq |R_1 \bowtie R_2| \leq |R_1|$
- Se il join coinvolge una chiave B di R_2 e un vincolo di integrità referenziale tra attributi di R_1 e la chiave di R_2 , allora il numero di tuple è pari a $|R_1 \bowtie R_2| = |R_1|$

Il **join esterno** estende, con valori nulli, le tuple che verrebbero tagliate fuori da un **join interno**. Esistono tre versioni:

- Sinistro ($R_1 \ltimes R_2$)**: mantiene tutte le tuple del primo operando, estendendole con valori nulli se necessario.

Impiegati		Reparti	
Impiegato	Reparto	Reparto	Capo
Rossi	A	B	Mori
Neri	B	C	Bruni
Bianchi	B		

Impiegati $\ltimes_{\text{LEFT}}$ Reparti		
Impiegato	Reparto	Capo
Neri	B	Mori
Bianchi	B	Mori
Rossi	A	NULL

- Destro ($R_1 \rtimes R_2$)**: mantiene tutte le tuple del secondo operando, estendendole con valori nulli se necessario.

Impiegati		Reparti	
Impiegato	Reparto	Reparto	Capo
Rossi	A	B	Mori
Neri	B	C	Bruni
Bianchi	B		

Impiegati $\rtimes_{\text{RIGHT}}$ Reparti		
Impiegato	Reparto	Capo
Neri	B	Mori
Bianchi	B	Mori
NULL	C	Bruni

- Completo ($R_1 \bowtie R_2$)**: mantiene tutte le tuple di entrambi gli operandi, estendendole con valori nulli se necessario.

Impiegati		Reparti	
Impiegato	Reparto	Reparto	Capo
Rossi	A	B	Mori
Neri	B	C	Bruni
Bianchi	B		

Impiegati $\bowtie_{\text{FULL}}$ Reparti		
Impiegato	Reparto	Capo
Neri	B	Mori
Bianchi	B	Mori
Rossi	A	NULL
NULL	C	Bruni

- **Prodotto cartesiano**: effettua un join naturale su relazioni **senza attributi in comune** e contiene sempre un numero di tuple pari al prodotto delle cardinalità degli operandi (le tuple sono tutte combinabili).
Perciò, concatena tuple non necessariamente correlate dal punto di vista semantico ed è più utile se fatto seguire da una **selezione** $\sigma_{\text{Condizione}}(R_1 \times R_2)$.
- **Theta join**: è il prodotto cartesiano seguito da una selezione, ovvero $R_1 \bowtie_{\text{Condizione}} R_2$. La condizione è spesso una congiunzione di atomi di confronto $A_1 \theta A_2$, dove θ è uno degli operatori di confronto ($=, >, <, \dots$).
Se l'operatore di confronto è sempre l'uguaglianza, allora si ha un **equi join**.
- **Self join**: effettua un join naturale sulla stessa relazione solo dopo aver effettuato una **ridenominazione**.

Genitori	
Genitore	Figlio
Luca	Anna
Maria	Anna
Giorgio	Luca
Silvia	Maria
Enzo	Maria

$\rho_{\text{Nonno, Genitore} \leftarrow \text{Genitore, Figlio}}(\text{Genitore})$

Nonno	Genitore
Luca	Anna
Maria	Anna
Giorgio	Luca
Silvia	Maria
Enzo	Maria

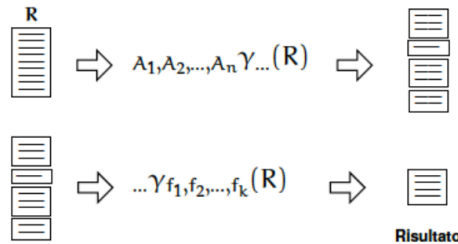
È possibile effettuare **composizioni di operatori**, come selezione e proiezione $\pi_{\text{Matricola, Cognome}} (\sigma_{\text{Stipendio} > 50} (\text{Impiegati}))$

Le relazioni sono insiemi e i risultati devono essere relazioni. È possibile applicare l'unione, l'intersezione e la differenza solo a relazioni definite sugli **stessi attributi** (nome e tipo), ovvero possono operare solo su tuple uniformi.

Il **raggruppamento** $\gamma_{\{A_i\}} \gamma_{\{f_i\}}(R)$ è una relazione calcolata come segue:

1. Si partizionano le tuple di R mettendo nello stesso gruppo tutte le tuple con valori uguali agli A_i
2. Si calcolano le espressioni f_i per ogni gruppo.
3. Per ogni gruppo si restituisce una sola tupla con attributi i valori degli A_i e delle espressioni f_i

Dove gli A_i sono attributi di R e f_i sono espressioni che usano funzioni di aggregazione ($\min$, $\max$, $\sum$, ...).



Ad esempio, per ogni candidato si vuole ottenere il numero degli esami, il voto minimo, massimo e medio:

$\{\text{Candidato}\} \gamma_{\{\text{count}(*), \min(\text{Voto}), \max(\text{Voto}), \text{avg}(\text{Voto})\}}(\text{Esami})$

Materia	Candidato	Voto	Docente
DA	1	20	10
LFC	2	30	20
MTI	1	30	30
LP	2	20	40

→

Materia	Candidato	Voto	Docente
DA	1	20	10
MTI	1	24	30
LFC	2	30	20
LP	2	20	40

↙

Candidato	Count(*)	min(Voto)	max(Voto)	avg(Voto)
1	2	20	24	22
2	2	20	30	25

Le **trasformazioni algebriche** sono basate su regole di equivalenza tra espressioni algebriche e consentono di scegliere diversi ordini di join e di anticipare proiezioni e restrizioni. Alcune trasformazioni algebriche sono:

- **Atomizzazione delle selezioni** ($\sigma_{F_1 \wedge F_2}(E) = \sigma_{F_1}(\sigma_{F_2}(E))$): una selezione congiuntiva può essere sostituita da una cascata di selezioni atomiche.
- **Idempotenza delle proiezioni** ($\pi_X(E) = \pi_X(\pi_{XY}(E))$): una proiezione può essere trasformata in una cascata di proiezioni che eliminano i vari attributi in fasi diverse se E è definita su un insieme di attributi che contiene sia X che Y (impatta sulla leggibilità della query, non sull'efficienza).
- **Anticipazione della selezione rispetto al join** ($\sigma_F(E_1 \bowtie E_2) = E_1 \bowtie \sigma_F(E_2)$): vale se F fa riferimento solo ad attributi di E_2 . Aumenta l'efficienza della query, dato che la selezione riduce il numero delle tuple di E_2 prima del join.
- **Anticipazione della proiezione rispetto al join** ($\pi_{X_1 Y_2}(E_1 \bowtie E_2) = E_1 \bowtie \pi_{Y_2}(E_2)$): vale se E_1 ed E_2 sono definite rispettivamente su X_1 e X_2 , se $Y_2 \subseteq X_2$ e se gli attributi in $X_2 - Y_2$ non sono coinvolti nel join (impatta sulla leggibilità, non sull'efficienza).
- **Distributività della selezione rispetto all'unione** ($\sigma_F(E_1 \cup E_2) = \sigma_F(E_1) \cup \sigma_F(E_2)$)
- **Distributività della selezione rispetto alla differenza** ($\sigma_F(E_1 - E_2) = \sigma_F(E_1) - \sigma_F(E_2)$)
- **Distributività della proiezione rispetto all'unione** ($\pi_X(E_1 \cup E_2) = \pi_X(E_1) \cup \pi_X(E_2)$)
- **Non distributività della proiezione rispetto alla differenza** ($\pi_A(R_1 - R_2) = \pi_A(R_1) - \pi_A(R_2)$): vale se R_1 e R_2 sono definite sull'insieme di attributi $X = AB$ e contengono tuple uguali su A e diverse su B .
- **Inglobamento selezione in un prodotto cartesiano a formare un join** ($\sigma_F(R_1 \bowtie R_2) = R_1 \bowtie_F R_2$)
- Altre equivalenze:
 - $\sigma_{F_1 \vee F_2}(R) = \sigma_{F_1}(R) \cup \sigma_{F_2}(R)$
 - $\sigma_{F_1 \wedge F_2}(R) = \sigma_{F_1}(R) \cap \sigma_{F_2}(R)$
 - $\sigma_{F_1 \wedge \neg F_2}(R) = \sigma_{F_1}(R) - \sigma_{F_2}(R)$

Le proprietà **commutativa** e **associativa** valgono per tutti gli operatori binari, tranne per la differenza.

Due operatori algebrici non insiemistici sono:

- **La proiezione multi-insiemistica**, senza eliminazione dei duplicati ($\pi_{\{A_i\}}^b(R)$)
- **L'ordinamento** ($\tau_{\{A_i\}}(R)$)

Il **calcolo relazionale** è un linguaggio che permette di definire il risultato di un'interrogazione come l'insieme delle tuple che soddisfano una certa condizione ϕ . Ad esempio:

L'insieme delle matricole degli studenti che hanno superato qualcuno degli esami elencati nella relazione *Materie*(Materia), si può definire come:

$$\{t.Matricula \mid t \in \text{Studenti}, \exists m \in \text{Materie}, \exists e \in \text{ProveEsami}, e.Candidato = t.Matricula \wedge e.Materia = m.Materia\}$$

Che è equivalente a

$$\pi_{Matricula}(\text{Studenti} \bowtie_{Matricula = Candidato} (\text{ProveEsami} \bowtie \text{Materie}))$$

L'**interrogazione** di una base di dati è uno degli aspetti più importanti del **linguaggio SQL**. I comandi di interrogazione (**query**) permettono di effettuare una ricerca dei dati presenti nel database che soddisfano particolari condizioni richieste dall'utente.

Nei linguaggi di interrogazione di basi di dati è importante distinguere tra:

- **Data Manipulation Language (DML)**: interrogazione e aggiornamento di istanze delle basi di dati.

```
SELECT Insegnamento
FROM Orario
WHERE Docente = "Mario Rossi"
```

- **Data Definition Language (DDL)**: definizione di schemi (logici, esterni e fisici) e altre operazioni.

```
CREATE TABLE Orario (
    Insegnamento CHAR(20),
    Docente CHAR(20),
    Aula CHAR(20)
)
```

Le query vengono effettuate mediante il comando **SELECT**, la cui sintassi è:

```
SELECT ListaAttributi          SELECT [DISTINCT] Attributo {, Attributo}
FROM ListaTabelle             or FROM Tabella [ID] {, Tabella [ID]}
[WHERE Condizione]            [WHERE Condizione]
```

È necessario specificare:

- La lista degli attributi interessati (**target list**), ovvero scegliere alcuni attributi delle tabelle considerate. Il comando **SELECT** implementa l'operazione di **proiezione π** dell'algebra relazionale.

```
* Attributi ::= *
| Expr [[AS] Nuovonome] {, Expr [[AS] Nuovonome]}

* Expr ::= [Ide.]Attributo | Const
| ( Expr ) | [-] Expr [Op Expr]
| COUNT(*)
| AggrFun ( [DISTINCT] [Ide.]Attributo )

* e AS x: dà un nome alla colonna di e

* AggrFun ::= SUM | COUNT | AVG | MAX | MIN
```

Per visualizzare tutti gli attributi (le colonne) di una tabella si utilizza il simbolo **asterisco** (**SELECT ***).

- La clausola **FROM** per stabilire in quali tabelle sono contenuti i dati richiesti. Seleziona le tabelle della base di dati da cui estrarre le informazioni. **In presenza di due tabelle in *ListaTabelle*** e insieme alla clausola **WHERE** (che seleziona quali righe delle tabelle accoppiare), **implementa il theta join**.
- La clausola **WHERE** per esprimere le condizioni che i dati richiesti devono soddisfare. Seleziona le righe della tabella che soddisfano la *Condizione* e implementa la **selezione σ** dell'algebra relazionale. La *Condizione* è una condizione booleana che fa uso di:

- Operatori di confronto: =, <, >, ≠, ≤, ≥
- Connettori logici: **AND**, **OR**, **NOT**
- Operatori (presenti anche in forma negativa, ovvero preceduti da **NOT**):
 - **BETWEEN**: seleziona le righe con attributi in un particolare range.
Ad esempio, ... **BETWEEN 10 AND 20**
 - **IN**: seleziona le righe che hanno un attributo che assume valori contenuti in una lista.
Ad esempio, ... **IN ('IIA', 'CN', 'CC', 'BD')**
 - **LIKE**: effettua ricerche wildcard (simbolo jolly) di una stringa di valori. Le condizioni di ricerca possono contenere letterali, caratteri o numeri.

Esistono due tipi di wildcard:

- %: indica zero o più caratteri.
- _: indica un carattere.

Ad esempio, ... *LIKE 'a'*

Nel caso in cui uno dei simboli coinvolti nel pattern matching sia % o _, è necessario scegliere un **simbolo escape**, ovvero un simbolo non ammesso tra i simboli della stringa. Scelto il simbolo, ad esempio #, lo si precede al simbolo % o _ e si specifica che tale simbolo è quello escape.

Ad esempio, ... *LIKE 'C#_F%' ESCAPE '#'*

- **IS [NOT] NULL**: verifica se il contenuto di un operando [non] è *NULL*.

Ad esempio, ... *IS NULL*

Una query considera il prodotto cartesiano tra le tabelle in *ListaTabelle* (join) e tra queste seleziona solo le righe che soddisfano la *Condizione* (selezione); infine, valuta le espressioni specificate nella target list *ListaAttributi* (proiezione).

Perciò, la *SELECT* implementa gli operatori di proiezione, selezione e join dell'algebra relazionale.

L'**alias** permette di dare a una colonna un nome diverso rispetto a quello utilizzato nella definizione della tabella. Implementa l'operazione di **ridenominazione ρ** dell'algebra relazionale. Può essere utilizzata opzionalmente la **keyword AS** tra il nome della colonna e l'alias, ma è sufficiente inserire uno spazio tra il nuovo nome e quello originale della tabella (in presenza di spazi nel nuovo nome, sono necessarie le **virgolette** ").

I valori *NULL* rappresentano l'**assenza di informazione** (valore sconosciuto, valore non disponibile o attributo non applicabile); perciò, ciascun valore *NULL* è considerato **diverso** dagli altri valori *NULL*. Il confronto logico con un valore *NULL* restituisce *SCONOSCIUTO (UNKNOWN)*, indicato con *S*.

NOT		OR	V	F	S	AND	V	F	S
V	F	V	V	V	V	V	V	F	S
F	V	F	V	F	S	F	F	F	F
S	S	S	V	S	S	S	S	F	S

È possibile realizzare **espressioni nella target list**. Ad esempio:

- Evidenziare i cognomi degli impiegati e il loro stipendio aumentato del 20%.
- Rinominare la colonna corrispondente al salario aumentato con la denominazione "Aumento".

Select cognome, stip *120/100 Aumento
FROM Impiegato

IMPIEGATO

Nome	Cognome	Stip
Mario	Rossi	1200
Luigi	Verdi	1130
Maria	Bianchi	1450
Luisa	Gialli	1300

Cognome	Aumento
Rossi	1440
Verdi	1356
Bianchi	1740
Gialli	1560

Se un argomento è *NULL*, lo è anche l'intera espressione.

È possibile ordinare il risultato di una *SELECT*. L'ordinamento può essere effettuato in base a un attributo e può essere crescente o decrescente. La sintassi è:

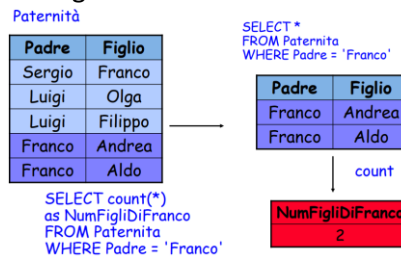
...
WHERE Condizioni
ORDER BY Attributo [ASC/DESC]

Perciò, le righe vengono ordinate in base al campo *Attributo* in maniera crescente (*ASC*, default) o decrescente (*DESC*).

È possibile ordinare i dati in base a un certo attributo, in un certo modo, e riordinare i dati che coincidono su quell'attributo in base a un altro. Ad esempio, *ORDER BY Cognome ASC, Nascita DESC*.

Nella target list è possibile utilizzare i seguenti **operatori aggregati**, i quali restituiscono una tabella costituita da un singolo valore scalare:

- **COUNT**: restituisce il numero di righe della tabella o il numero di valori di un particolare attributo. Viene applicato al risultato dell'interrogazione.



Mediante le seguenti specifiche è possibile contare:

- **(*)**: tutte le righe selezionate.
- **ALL** (default): tutti i valori **non nulli** delle righe selezionate.
- **DISTINCT**: tutti i valori **non nulli distinti** delle righe selezionate.
- **MAX** e **MIN**: calcolano il maggiore e il minore degli elementi di una colonna. L'argomento può essere un'espressione aritmetica.
- **AVG**: calcola la media dei valori non nulli di una colonna.

Mediante le seguenti specifiche è possibile calcolare la media tra:

- **ALL** (default): tutti i valori **non nulli** delle righe selezionate.
- **DISTINCT**: tutti i valori **non nulli distinti** delle righe selezionate.
- **SUM**: calcola la somma dei valori di una colonna.

Mediante le seguenti specifiche è possibile sommare:

- **ALL** (default): tutti i valori **non nulli** delle righe selezionate.
- **DISTINCT**: tutti i valori **non nulli distinti** delle righe selezionate.

In generale, i valori **NULL** vengono ignorati; perciò, se in una riga il valore di **Prezzo** = **NULL** può succedere che **SUM(Prezzo + IVA)** sia diverso da **SUM(Prezzo) + SUM(IVA)**.

Non è possibile effettuare, nella stessa **SELECT**, una proiezione su attributi della tabella e operatori aggregati.

QUERY ERRATA <pre>SELECT Nome, MAX(Reddito) FROM Persone</pre>	QUERY CORRETTA <pre>SELECT MIN(Eta), AVG(Reddito) FROM Persone</pre>
--	--

La clausola **GROUP BY** permette raggruppare le righe i cui valori coincidono su un certo attributo. In questo modo è possibile calcolare operatori aggregati non su tutta la tabella, ma una volta per gruppo.

Per esempio, vogliamo sapere la media degli stipendi degli impiegati per ogni dipartimento. In tal caso si può utilizzare la clausola **GROUP BY**.

Query:

```
SELECT Dipart, AVG(stipendio)
FROM Impiegati
GROUP BY Dipart
```

Dipart	AVG(stipendio)
Produzione	1330
Amministrazione	1505
Distribuzione	1810
Direzione	2500

La query viene eseguita senza l'operatore aggregato **AVG** e senza la clausola **GROUP BY**. Successivamente, il risultato viene diviso in sottoinsiemi aventi gli stessi valori per gli attributi indicati nella clausola **GROUP BY**. Infine, l'operatore aggregato **AVG** viene calcolato su ogni sottoinsieme.

Dipart	Stipendio
Amministrazione	1560
Produzione	1250
Distribuzione	1750
Produzione	1340
Direzione	2500
Distribuzione	1870
Produzione	1400
Amministrazione	1450

→

Dipart	Stipendio
Amministrazione	1560
Amministrazione	1450
Distribuzione	1750
Distribuzione	1870
Direzione	2500
Produzione	1340
Produzione	1400
Produzione	1250

→ **AVG** →

Dipart	AVG(stipendio)
Amministrazione	1505
Distribuzione	1810
Direzione	2500
Produzione	1330

Un raggruppamento dev'essere effettuato su tutti gli elementi della target list che **non** sono operatori aggregati. Questo perché, nel risultato, deve apparire una riga per ogni gruppo. Perciò, gli attributi **non aggregati** nella target list **devono essere inclusi** tra quelli citati nella clausola **GROUP BY**, mentre gli attributi aggregati non devono essere tra quelli raggruppati.

La clausola **HAVING** permette di applicare condizioni sul valore aggregato per ogni gruppo. Si utilizza solo **in presenza e dopo** di un **GROUP BY**.

Esempio: I dipartimenti la cui media degli stipendi è maggiore di 1700 euro

Select dipart, AVG(stipendio)
 FROM Impiegati
 Group by Dipart
 HAVING AVG(stipendio)>1700

Dipart	AVG(stipendio)
Amministrazione	1505
Distribuzione	1810
Direzione	2500
Produzione	1330

Dipart	AVG(stipendio)
Distribuzione	1810
Direzione	2500

HAVING AVG(stipendio)>1700

In generale, si utilizza:

- **WHERE** se la condizione coinvolge un **attributo**.
- **HAVING** se la condizione coinvolge un **operatore aggregato**.

Spesso, nel riferimento alle colonne selezionate nel join, è necessario specificare da quale tabella la colonna è stata estratta, così da evitare ambiguità. Ciò è possibile mediante la sintassi **NomeTabella.NomeColonna**. Di solito, in una **SELECT** che definisca una join sono necessarie **ridenominazioni**:

- Nel **prodotto cartesiano** (ridenominazione delle tabelle coinvolte).
- Nella **target list** (ridenominazione degli attributi coinvolti).

```

SELECT X.A1 AS B1, Y.A2 AS B2 ...
FROM Tab1 X, Tab2 Y, Tab1 Z
WHERE X.A2 = Y.A3 AND ...
  
```

Il **cross join** implementa il prodotto cartesiano e si realizza mediante una **SELECT** su due o più tabelle, senza specificare nessuna condizione di join.

```

SELECT Categorie.*, Fabbriche.*
FROM Categorie, Fabbriche
  
```

Se due tabelle contengono dati in comune, possono essere correlate mediante un'operazione di **join**. Le colonne delle due tabelle che creano la correlazione **rappresentano la stessa entità**, ovvero i loro valori appartengono allo **stesso dominio**. Solitamente, le colonne delle due tabelle sono legate da un **vincolo di chiave esterna**, ma ciò non è obbligatorio.

L' **equi join** tra due tabelle è una terza tabella le cui righe sono solo quelle ottenute dal prodotto cartesiano delle righe delle tabelle di partenza, i cui **valori delle colonne** espresse dalla condizione di join sono **uguali**.

In SQL viene realizzato mediante una **SELECT**, in cui

- Nella clausola **FROM** vengono indicate le tabelle coinvolte.
- Nella clausola **WHERE** viene espresso il collegamento tra le tabelle mediante la condizione di join.

```

SELECT Tab1.A1, Tab2.A4
FROM Tab1, Tab2
WHERE Tab1.A2 = Tab2.A3
  
```

A1	A2
A	B
C	D
E	F
G	H

A3	A4
F	I
D	L
D	M
B	N

A1	A4
A	N
C	L
C	M
E	I

L'**inner join** è un'operazione di join in cui la condizione non è necessariamente una condizione di uguaglianza.

Ad esempio:

```

SELECT Veicoli.*
FROM Modelli, Veicoli
WHERE Veicoli.ID_modello = Modelli.ID_modello AND CC_medio > Cilindrata
  
```

Il **self join** è un'operazione di join che mette in relazione una tabella con sé stessa. Questo si ottiene ridenominando due volte la tabella con due nomi diversi e trattando le due copie come se fossero di due tabelle diverse.

```
SELECT X.A1, Y.A4
FROM Tab1 X, Tab2 Y, Tab1 Z
WHERE X.A2 = Y.A3 AND X.A2 = Z.A1
```

Per effettuare il join di due tabelle è possibile utilizzare una **forma più esplicita**, la cui sintassi è:

```
SELECT targa, Veicoli.Categoria, nome_cat
FROM Categorie JOIN Veicoli
ON Veicoli.Categoria = Categorie.Cod_cat
```

Il **natural join** è un'operazione di join effettuata su tutte le colonne con lo stesso nome in entrambe le tabelle.

```
SELECT listaAttributi
FROM Tab1 NATURAL JOIN Tab2
```

La clausola **USING**, seguita dall'elenco degli attributi coinvolti nella condizione, permette di effettuare una join in cui la condizione riguarda solo una o alcune coppie di attributi con lo stesso nome.

```
SELECT Nome_mod
FROM Modelli JOIN Veicoli
USING (Cod_mod)
WHERE targa="ZX2345"
```

L'**outer join**, a differenza dell'equi join, mantiene i valori di dati in comune che non sono presenti in tutte le tabelle coinvolte. Ne esistono tre tipi:

```
Select lista_attributi
From Tab1 LEFT [OUTER] JOIN Tab2

Select lista_attributi
From Tab1 RIGHT [OUTER] JOIN Tab2

Select lista_attributi
From Tab1 FULL [OUTER] JOIN Tab2
```

L'outer join può essere applicato usando la seguente sintassi:

- {LEFT | RIGHT | FULL} [OUTER] JOIN + ON <predicato>
- {LEFT | RIGHT | FULL} [OUTER] JOIN + USING <colonne>

Una **subquery** è un comando *SELECT* racchiuso tra parentesi tonde e inserito all'interno di un comando SQL, come un'altra *SELECT*. Le subquery possono essere utilizzate:

- In espressioni di confronto.
- In espressioni di confronto quantificato.
- In espressioni *IN*.
- In espressioni *EXISTS*.
- Nel calcolo di espressioni.

Esistono tre tipologie di subquery:

- Subquery **scalare**: comando *SELECT* che restituisce un solo valore.
- Subquery **di colonna**: comando *SELECT* che restituisce una colonna.
- Subquery **di tabella**: comando *SELECT* che restituisce una tabella.

Mediante una subquery è possibile utilizzare **contemporaneamente** funzioni di gruppo e funzioni su singole righe, cosa non possibile con le semplici *SELECT*.

```
Select *
From Veicoli
Where Cilindrata > (select AVG(Cilindrata)
From Veicoli)
```

Inoltre, le subquery possono essere **nidificate**:

```
Select * From Modelli
Where num_versioni > (Select min (num_versioni)
From Modelli
Where cod_mod IN (select cod_mod
From veicoli
Where cod_combust='01' ) )
```

La forma nidificata è **meno dichiarativa**, ma **più leggibile**, dato che richiede meno variabili; essa può essere **combinata** con quella piana.

Le subquery **non possono contenere operatori insiemistici**. È possibile ovviare a questo problema mediante una forma alternativa per l'intersezione e la differenza, ma non sempre per l'unione.

Inoltre, le **regole di visibilità** sono simili a quelle delle procedure nei linguaggi di programmazione, dove:

- Non è possibile fare riferimenti a variabili definite in blocchi più interni.
- Se un nome di una variabile è omissso, si assume come riferimento la variabile più "vicina".
- In un blocco è possibile fare riferimento a variabili definite in blocchi più esterni.

Il confronto con il risultato di una query nidificata può essere **basato su più attributi**:

```
SELECT *
FROM Studenti S
WHERE (Nome, Cognome) IN (SELECT Nome, Cognome
FROM Studenti S2
WHERE S2.Matricola <> S.Matricola)
```

• • • Condizione importante!

Perciò, le query nidificate possono essere meno dichiarative, ma spesso sono **più facilmente interpretabili**, dato che possono essere suddivise in blocchi più semplici da interpretare. **Query nidificate complesse** possono essere di difficile comprensione, soprattutto utilizzando molte variabili comuni tra blocchi diversi. L'utilizzo di variabili deve rispettare le **regole di visibilità**, ovvero una variabile può essere usata solo all'interno dello stesso blocco e in un blocco più interno.

Tutte le interrogazioni su di un'associazione multivalore devono essere quantificate. Esistono due tipi di quantificazione:

- **Universale/totale**: "Gli studenti che hanno preso sempre 30".
La quantificazione **esistenziale negata** corrisponde a quella universale: "Gli studenti che **non** hanno preso almeno un 30".
- **Esistenziale**: "Gli studenti che hanno preso almeno un 30".
La quantificazione **universale negata** corrisponde a quella esistenziale: "Gli studenti che **non** hanno preso sempre 30".

In SQL, le condizioni permettono anche il confronto tra un attributo e il risultato di una subquery che restituisce una colonna o una tabella, mediante dei **predicati**:

- Per un **operatore scalare**, (*ANY* / *ALL*) *SubSELECT...*, dove:
 - *ANY* è vero se **almeno uno** dei valori restituiti dalla query successiva soddisfa la condizione.
 - *ALL* è vero se **tutti** i valori restituiti dalla query successiva soddisfano la condizione.
- Per il quantificatore universale, *EXISTS SubSELECT...*, dove *EXISTS* è vero se la query successiva restituisce **almeno una** tupla. Nel caso di *NOT EXISTS*, è vero se la query successiva **non** restituisce almeno una tupla.

Di seguito, alcune regole sull'utilizzo delle subquery di confronto quantificato:

- Una subquery dev'essere una subquery di colonna.
- Una subquery dev'essere inserita **dopo** l'operatore di confronto quantificato.
- Non è possibile confrontare due subquery.
- In una subquery non è possibile utilizzare le clausole *HAVING* e *GROUP BY*.

Per rendere più flessibile la clausola *EXISTS*, così da rendere le condizioni della subquery dipendenti dalla tupla della query principale (la query esterna), è possibile introdurre un legame tra la query e la subquery (**query correlate**). Per fare ciò, si definisce un **alias** nella query esterna da utilizzare nella subquery. Inoltre, la query più interna può usare variabili di quella esterna e l'interrogazione interna viene eseguita **una volta per ogni tupla** dell'interrogazione esterna.

Così facendo, il predicato *EXISTS* permette di effettuare il controllo sull'esistenza di righe che soddisfano specifiche condizioni; in questo caso, la subquery è una **subquery di tabella**.

Mediante il predicato *NOT EXISTS* è possibile verificare l'assenza di righe che corrispondono a una determinata condizione.

Le forme $= ANY$ (equivalente a IN e a $EXISTS$) e $\neq ALL$ (equivalente a $NOT IN$) forniscono un modo alternativo per realizzare l'intersezione e la differenza dell'algebra relazionale.

Le subquery, oltre a essere utilizzate all'interno della clausola *WHERE*, possono essere utilizzate nel **calcolo di espressioni**, ovvero per definire delle colonne.

```
Select Targa, Cilindrata, Cilindrata - (Select min (Cilindrata)
From Veicoli) Differenza
From Veicoli
```

Le operazioni di **unione**, **intersezione** e **differenza** lavorano sulle tabelle come se fossero **insiemi di righe**, ovvero i **duplicati vengono eliminati** anche dalle proiezioni (tranne se viene utilizzata l'opzione *ALL*):

- **UNION**: utilizza come operandi le due tabelle ottenute mediante due *SELECT* e restituisce una nuova tabella contenente **tutte le righe della prima e della seconda tabella**.
In presenza di righe duplicate, l'operatore *UNION* ne mantiene **una sola copia**, tranne se specificata l'opzione *ALL* (indica la volontà di **mantenere i duplicati**).

```
SELECT padre, figlio
FROM paternita
UNION
SELECT madre, figlio
FROM maternita
```

- **INTERSECT**: utilizza come operandi due tabelle ottenute mediante due *SELECT* e restituisce una nuova tabella contenente **le righe comuni** dalle due tabelle iniziali.

```
SELECT ...
INTERSECT [ALL]
SELECT ...
```

- **EXCEPT**: utilizza come operandi due tabelle ottenute mediante due *SELECT* e restituisce una nuova tabella contenente **tutte le righe della prima che non si trovano nella seconda**.

```
SELECT Nome
FROM Impiegato
EXCEPT
SELECT Cognome as Nome
FROM Impiegato
```

In presenza di righe duplicate, questi operatori mantengono **una sola copia**, tranne se specificata l'opzione *ALL* (indica la volontà di **mantenere i duplicati**).

Il **Data Manipulation Language (DML)** è il linguaggio SQL che permette di **inserire**, **modificare** e **cancellare** i dati nella base di dati, ma anche di **interrogare** la base di dati, ovvero estrarne i dati.

Le istruzioni per inserire, modificare e cancellare i dati sono:

- **INSERT**: permette di inserire uno o più dati in una tabella. Di seguito la sintassi:

```
INSERT INTO nome tabella
[(ListaAttributi)] VALUES (ListaDiValori) |
SQLSelect
```

Ad esempio:

```
INSERT INTO Esami
(Corso, Matricola, Voto) VALUES ("IIA", "123456", 27)
```

Ai **valori non attribuiti** viene assegnato *NULL*, tranne se specificato un diverso **valore di default**. Inoltre, l'inserimento fallisce se *NULL* non è permesso per gli attributi mancanti.

Non specificare gli attributi equivale a specificare **tutte le colonne** della tabella ed è importante rispettare l'**ordine degli attributi**.

È possibile effettuare un'*INSERT* prendendo i dati da un'altra tabella mediante una *SELECT*. In questo modo è possibile effettuare tanti inserimenti simultaneamente.

```
INSERT INTO Indirizzi_Studenti (Indirizzo, Telefono)
SELECT Indirizzo, Telefono
FROM Studenti
```

- **UPDATE**: permette di aggiornare le righe di una tabella. Di seguito la sintassi:

```
UPDATE Tabella
SET Attributo = Espr
WHERE Condizione
```

Ad esempio:

```
UPDATE Persone
SET Reddito = Reddito * 1.1
WHERE Eta < 30
```

- **DELETE**: permette di cancellare le righe di una tabella. Di seguito la sintassi:

```
DELETE FROM nome_tabella
[WHERE Condizione]
```

Ad esempio:

```
DELETE FROM Esami
WHERE Matricola = "123456"
```

Per eliminare un elemento è necessario individuare quale, mediante la clausola *WHERE*, dove viene stabilita una condizione che individua **uno o più elementi da cancellare**. Spesso un elemento può essere individuato mediante il suo valore nella chiave primaria.

L'operazione di cancellazione **non è reversibile**; inoltre, se la clausola *WHERE* viene omessa, viene cancellato **l'intero contenuto della tabella**, mentre lo schema non viene modificato.

La condizione della *DELETE* può essere una condizione di *SELECT*. In questo modo, è possibile cancellare più righe con un'unica istruzione, a patto che le righe soddisfino la condizione.

```
DELETE FROM Esami
WHERE Matricola NOT IN
(SELECT Matricola FROM Studenti)
```

Il **Data Definition Language (DDL)** è il linguaggio SQL che permette la **creazione, modifica e cancellazione** delle tabelle, dei domini e degli altri oggetti della base di dati, al fine di definirne il suo schema logico.

L'istruzione **CREATE TABLE** permette di definire le tabelle, corrispondenti alle relazioni dell'algebra lineare. Essa definisce uno schema di relazione e ne crea un'istanza vuota, specificandone attributi, domini e vincoli. Una volta creata, la tabella è pronta per l'inserimento dei dati che dovranno soddisfare i vincoli imposti. Di seguito la sintassi:

```
CREATE TABLE <nome_tabella>
( nome_colonna_1 tipo_colonna_1 clausola_default_1 vincolo_di_colonna_1,
  nome_colonna_2 tipo_colonna_2 clausola_default_2 vincolo_di_colonna_2,
  .....
  nome_colonna_k tipo_colonna_k clausola_default_k vincolo_di_colonna_k,
  vincoli di tabella
)
```

Ad esempio:

```
CREATE TABLE IMPIEGATO (
  Matricola CHAR(6) PRIMARY KEY,
  Nome CHAR(20) NOT NULL,
  Cognome CHAR(20) NOT NULL,
  Dipart CHAR(15),
  Stipendio NUMERIC(9) DEFAULT 0,
  FOREIGN KEY(Dipart) REFERENCES Dipartimento(NomeDip),
  UNIQUE (Cognome,Nome)
)
```

La **visualizzazione dello schema di una tabella**, dopo essere stata creata, può essere ottenuta mediante il comando **DESCRIBE**. Nel caso dell'esempio precedente, con **DESCRIBE Impiegato**, si ottiene:

```
IMPIEGATO
+-----+-----+-----+-----+-----+
| Matricola | Nome | Cognome | Dipart | Stipendio |
+-----+-----+-----+-----+-----+
```

I tipi più comuni per i valori degli attributi di una tabella sono:

- *CHAR(n)*: stringhe di caratteri di lunghezza fissa *n*.
- *VARCHAR(n)*: stringhe di caratteri di lunghezza variabile di, al massimo, *n* caratteri.
- *INTEGER*: interi con la dimensione uguale alla parola di memoria standard dell'elaboratore.
- *REAL*: numeri reali con dimensione uguale alla parola di memoria standard dell'elaboratore.
- *NUMBER(p, s)*: numeri con *p* cifre, di cui *s* decimali.
- *FLOAT(p)*: numeri binari in virgola mobile, con almeno *p* cifre significative.
- *DATE*: valori che rappresentano istanti di tempo (*TIME*) o solo date.

Ciò che viene creato mediante l'istruzione *CREATE* può essere cambiato con l'istruzione *ALTER* ed eliminato con l'istruzione *DROP*.

Con l'istruzione **ALTER TABLE** è possibile modificare una tabella e:

- **Aggiungere una colonna**, mediante la sintassi:

```
ALTER TABLE nome_tabella
ADD [COLUMN] nome_col tipo_col default_col vincolo_col
```

Ad esempio:

```
ALTER TABLE impiegato
ADD COLUMN nomecapo varchar(20) default 'Rossi' not null
```

La nuova colonna viene inserita come **ultima** colonna della tabella, a meno che non si inserisca la specifica **ADD COLUMN ... [FIRST | AFTER CoLonna]**, dove:

- **FIRST** la aggiunge come **prima** colonna.
- **AFTER** la aggiunge **subito dopo CoLonna**.

È sempre possibile aggiungere una colonna se non viene specificato **NOT NULL**.

- **Eliminare una colonna**, mediante la sintassi:

```
ALTER TABLE nome_tabella
DROP COLUMN nome_colonna {RESTRICT/CASCADE}
```

Ad esempio:

```
ALTER TABLE Impiegato      ALTER TABLE impiegato
Drop column dipart restrict  Drop column dipart cascade
```

Le opzioni **RESTRICT** e **CASCADE** sono obbligatorie e alternative (ne va scelta almeno e solo una):

- **RESTRICT**: se un'altra tabella ha un vincolo di integrità referenziale (chiave esterna) con la colonna da eliminare, l'esecuzione fallisce.
- **CASCADE**: eliminando la colonna vengono eliminate tutte le dipendenze logiche di altre colonne dello schema da questa.

- **Modificare una colonna**, mediante la sintassi:

```
ALTER TABLE nome_tabella MODIFY
nome_colonna tipo_col default_col vincoli_col
```

Ad esempio:

```
ALTER TABLE impiegato MODIFY
nome varchar(30) not null
```

- **Assegnare un valore di default**, mediante la sintassi:

```
ALTER TABLE nome_tabella
ALTER [COLUMN] nome_colonna
SET DEFAULT valore_default
```

- **Eliminare un valore di default**, mediante la sintassi:

```
ALTER TABLE nome_tabella
ALTER [COLUMN] nome_colonna
DROP DEFAULT
```

Una volta eliminato, il valore di default diventa **NULL**.

- **Aggiungere vincoli di tabella**, mediante la sintassi:

```
ALTER TABLE nome_tabella
ADD CONSTRAINT nome_vincolo vincolo_di_tabella
```

Ad esempio:

```
ALTER TABLE impiegato
ADD CONSTRAINT unique_const unique(nome, cognome)
```

È possibile aggiungere i seguenti vincoli:

- **Chiave primaria**: **ADD CONSTRAINT PK PRIMARY KEY (...)**
- **Chiave esterna**: **ADD CONSTRAINT FK FOREIGN KEY (...) REFERENCES ...**
- **Unicità**: **ADD CONSTRAINT ... UNIQUE (...)**
- **Controllo**: **ADD CONSTRAINT ... CHECK (...)**

- **Eliminare vincoli di tabella**, mediante la sintassi:

```
ALTER TABLE nome_tabella
DROP CONSTRAINT nome_vincolo {RESTRICT/CASCADE}
```

Le opzioni **RESTRICT** e **CASCADE** sono obbligatorie e alternative (ne va scelta almeno e solo una):

- **RESTRICT**: non permette di eliminare vincoli di unicità e di chiave primaria su una colonna se esistono vincoli di chiave esterna che si riferiscono a tale colonna.
- **CASCADE**: non opera la restrizione dell'opzione **RESTRICT**.

Per eliminare un vincolo è necessario che esso sia definito mediante un identificatore.

Con l'istruzione **DROP TABLE** è possibile eliminare una tabella; inoltre, è possibile specificare le opzioni:

- **RESTRICT**: se la tabella è utilizzata nella definizione di altri oggetti dello schema, la sua eliminazione viene impedita.
- **CASCADE**: vengono eliminate tutte le dipendenze degli altri oggetti dello schema da questa tabella.

I **vincoli di integrità** permettono di limitare i valori ammissibili per una determinata colonna di una tabella, in base a specifici criteri.

I **vincoli di integrità intrarelazionali** (non fanno riferimento ad altre relazioni) sono:

- **NOT NULL**
- **UNIQUE**, per definire le **chiavi della relazione**, non una chiave primaria.

Può essere utilizzato in due forme (due sintassi):

- Nella **definizione di un attributo**, se forma da solo la chiave; in questo caso, non è possibile avere due valori uguali in tale colonna.
- Come **elemento separato**.

Il vincolo di unicità può essere riferito a coppie o insiemi di attributi; questo significa che non possono esserci due righe per cui l'insieme dei valori corrispondenti agli attributi dell'insieme siano uguali. In questo caso, il vincolo viene dichiarato **dopo** aver dichiarato tutte le colonne, mediante un vincolo di tabella utilizzando l'istruzione **UNIQUE (ListaAttributi)**.

- **PRIMARY KEY**, per definire la chiave primaria (implica **NOT NULL**).

Può essere utilizzato in due forme (due sintassi):

- Nella **definizione di un attributo**, se forma da solo la chiave; in questo caso, non è possibile avere due valori uguali in tale colonna.
- Come **elemento separato**.

È simile al vincolo di unicità, ma definisce la **chiave primaria della relazione**, ovvero un attributo che individua univocamente un dato; perciò, implica sia il vincolo **UNIQUE** che il vincolo **NOT NULL**.

Una chiave primaria serve a identificare univocamente i soggetti del dominio; inoltre, permette il collegamento tra due tabelle.

Anche il vincolo di chiave primaria può essere definito su insiemi di elementi, utilizzando l'istruzione **PRIMARY KEY (ListaAttributi)**.

- **CHECK**, per controllare che una o più colonne soddisfino una condizione per ogni riga della tabella. Questo vincolo deve essere un'espressione booleana valutata usando i valori della colonna che vengono inseriti o aggiornati nella riga.

Può essere utilizzato in due forme (due sintassi):

- **Vincolo di riga**: può coinvolgere **solo** l'attributo su cui è definito.
- **Vincolo di tabella**: coinvolge **più** attributi.

Ad esempio, *Stipendio NUMERIC(9) DEFAULT 0 CHECK (Stipendio >= 0)*.

I **vincoli interrelazionali** vengono imposti quando gli attributi di due diverse tabelle devono essere messi in relazione. Questo permette di soddisfare l'esigenza di una base di dati di **non essere ridondante e di avere i dati sincronizzati**. Infatti, se due tabelle gestiscono gli stessi dati, è bene che di essi non ce ne siano più copie, sia allo scopo di **non occupare troppa memoria**, sia in modo che le modifiche fatte su dati uguali, utilizzati da due tabelle, siano **coerenti**.

I **vincoli interrelazionali REFERENCES e FOREIGN KEY** permettono di definire vincoli di integrità referenziale.

Possono essere utilizzati in due forme (due sintassi):

- Per **singoli** attributi, come **vincolo di colonna**.
Ad esempio, *Attributo REFERENCES Tabella (Colonna)*.
- Per **più** attributi, come **vincolo di tabella**.
Ad esempio, *FOREIGN KEY (Colonne) REFERENCES Tabella (Colonne)*.

Inoltre, è possibile definire politiche di **reazione alla violazione**, ovvero stabilire l'azione che il DBMS deve compiere quando si viola un vincolo in modifica o cancellazione.

Le reazioni alla violazione vengono dichiarate al momento della definizione dei vincoli di chiave esterna, mediante le istruzioni:

- **ON UPDATE**, le cui reazioni alla violazione sono:
 - **Impedire l'aggiornamento (NO ACTION)**: rifiuta gli aggiornamenti che violano l'integrità referenziale.
 - **Generare un aggiornamento a catena (CASCADE)**: le righe della tabella referente vengono impostati ai valori della tabella riferita.
 - **Assegnare valore NULL (SET NULL)**: assegna *NULL* ai valori della tabella referente.
 - **Assegnare il valore di default (SET DEFAULT)**: assegna il valore di default ai valori della tabella referente.
- **ON DELETE**, le cui reazioni alla violazione sono:
 - **Impedire la cancellazione (NO ACTION)**: azione di default che blocca la cancellazione delle righe della tabella riferita quando ci sono righe che dipendono da essa.
 - **Generare una cancellazione a catena (CASCADE)**: cancella tutte le righe dipendenti dalla tabella quando la corrispondente riga è cancellata dalla tabella riferita.
 - **Assegnare valore NULL (SET NULL)**: assegna *NULL* ai valori della colonna che hanno il vincolo di chiave esterna nella tabella quando la riga corrispondente viene cancellata dalla tabella riferita.
 - **Assegnare il valore di default (SET DEFAULT)**: assegna il valore di default ai valori della colonna che hanno il vincolo di chiave esterna nella tabella quando la riga corrispondente viene cancellata dalla tabella riferita.

Le viste, o viste logiche, possono essere definite come delle **tabelle virtuali**, i cui dati sono riaggregazioni dei dati contenuti nelle tabelle "fisiche" presenti nella base di dati (gli unici veri contenitori dei dati).

Le viste non contengono dati fisicamente diversi dai dati presenti nelle tabelle, ma forniscono una **diversa visione**, dinamicamente aggiornata, di quei dati.

Una vista appare all'utente come una normale tabella in cui può effettuare **interrogazioni** e, a seconda dei suoi privilegi, **modifiche** dei dati.

I vantaggi delle viste sono:

- **Una semplificazione della rappresentazione dei dati**: oltre ad assegnare un nome alla vista, è possibile cambiare il nome delle colonne.
- **Una convenienza** nello svolgere **serie di query complesse**.
- **Una protezione della base di dati**: le viste ad accesso limitato possono essere utilizzate per controllare le informazioni alle quali accede un certo utente della base di dati.
- **Una conversione** delle unità di misura e una **creazione di nuovi formati**.

Mentre le limitazioni sono:

- **L'impossibilità** di utilizzare gli **operatori booleani** *UNION*, *EXCEPT* e *INTERSECT*. (È possibile realizzare gli operatori *EXCEPT* e *INTERSECT* mediante una *SELECT*)
- **L'impossibilità** di utilizzare la clausola **ORDER BY**.

L'istruzione DDL per definire una vista ha la seguente sintassi:

```
CREATE VIEW NomeVista [(ListaAttributi)] AS SubSELECT
[WITH [LOCAL | CASCADE] CHECK OPTION]
```

I nomi delle colonne indicati in *ListaAttributi* sono i nomi assegnati alle colonne della vista, i quali corrispondono ordinatamente alle colonne elencate nella *SubSELECT*. Se questi nomi non sono specificati, le colonne della vista assumono gli stessi nomi di quelli della tabella a cui si riferisce.

Nonostante il contenuto di una vista sia dinamico, la struttura non lo è; perciò, se viene aggiunta una colonna alla tabella a cui la vista fa riferimento, questa nuova definizione **non si estende** alla vista, la quale conterrà le stesse colonne che aveva prima dell'inserimento della nuova colonna.

Una **visita di gruppo** è una visita in cui una delle colonne è una **funzione di gruppo**. In questo caso è obbligatorio assegnare un nome alla colonna della visita corrispondente alla funzione di gruppo e ridenominare la colonna relativa all'operatore aggregato.

(Una vista definita in base a una vista di gruppo è una vista di gruppo).

```
Create view A3 (cod_Fabbrica, numero_versioni) AS
Select cod_Fabbrica, sum(num_versioni)
From Modelli
Group by Cod_Fabbrica

Create view A4 AS
Select num_versioni
From A3
```

Per **eliminare una vista** si utilizza l'istruzione **DROP VIEW Nome {RESTRICT/CASCADE}**, dove:

- **RESTRICT** elimina la vista solo se non è riferita nella definizione di altri oggetti.
- **CASCADE** elimina la vista e tutte le dipendenze di altre definizioni dello schema da tale vista.

Quindi:

- Le tabelle create mediante **CREATE TABLE** sono **tabelle inizializzate**.
- Le viste create mediante **CREATE VIEW** sono **tabelle calcolate**.

Le tabelle delle viste sono interrogabili come le altre, ma **non possono essere modificate**; inoltre, deve esistere una **corrispondenza biunivoca** tra le righe della vista e le righe di una tabella di base, ovvero:

- La **SELECT** si effettua senza **DISTINCT**, ma solo con gli attributi.
- Il **FROM** si effettua su una sola tabella modificabile.
- Il **WHERE** si effettua senza **SubSELECT**.
- **GROUP BY** e **HAVING** non sono presenti nella definizione.

Possono esistere restrizioni sulle **SELECT** su viste definite usando **GROUP BY**.

I nuovi DBMS permettono di effettuare le operazioni **INSERT**, **UPDATE** e **DELETE** sulle viste, ma con **certe limitazioni** dovute alla definizione di vista stessa. L'aggiornamento di una vista è utile nel caso di un accesso ai dati controllato.

L'opzione **WITH CHECK OPTION**, messa alla fine della definizione della vista, assicura che le operazioni di inserimento e modifica dei dati effettuate utilizzando la vista soddisfino la clausola **WHERE** della subquery.

Una vista creata con l'opzione **WITH CHECK OPTION** implementa di default l'opzione **CASCADE**; perciò, se definita in termini di un'altra vista, il DBMS verifica che la nuova tupla inserita soddisfi le definizioni di entrambe le viste, indipendentemente dal fatto che l'altra vista sia stata a sua volta definita con l'opzione **WITH CHECK OPTION**.

Per alterare questo comportamento è necessario utilizzare l'opzione **WITH LOCAL CHECK OPTION**; in questo modo, il DBMS verifica che la tupla soddisfi la specifica della vista in cui viene inserita e quelle di tutte le viste da cui la vista dipende.

Dato che uno dei requisiti per la progettazione di una base di dati è la **normalizzazione dei dati**, le viste permettono di fornire all'utente i dati in una forma **più intuitiva**. Infatti, la forma normalizzata permette una corretta modellazione della realtà che la base di dati rappresenta, ma dal punto di vista dell'utente comporta una maggiore difficoltà di comprensione rispetto a una rappresentazione non normalizzata.

Un altro vantaggio delle viste riguarda l'**indipendenza logica** delle applicazioni e delle operazioni eseguite dagli utenti rispetto alla struttura logica dei dati; perciò, è possibile effettuare modifiche allo schema senza dover apportare modifiche alle applicazioni che utilizzano la base di dati.

Inoltre, le viste vengono anche utilizzate come una normale tabella per utilizzare in catena due diversi operatori aggregati, come la massima somma ($MAX(SUM(...))$), il minimo numero ($MIN(COUNT(...))$), ...

Un **trigger** definisce un'azione che la base di dati deve effettuare automaticamente su specifiche tabelle quando si verifica un determinato evento di manipolazione dei dati. Essi possono essere utilizzati per:

- **Migliorare l'integrità** referenziale dichiarativa.
- **Imporre regole complesse** legate all'attività della base di dati.
- **Effettuare revisioni** sulle modifiche dei dati.

L'esecuzione dei trigger è trasparente all'utente.

Gli **eventi** che si possono verificare sono i comandi DML (*INSERT*, *UPDATE* e *DELETE*) e DDL (*CREATE TABLE*, *CREATE VIEW*, ...); inoltre, anche gli aggiornamenti di specifiche colonne possono essere utilizzati come trigger di eventi.

I trigger a livello di riga vengono eseguiti **una volta per ciascuna riga** modificata in una transazione. Vengono utilizzati in applicazioni di revisione dei dati e si rivelano utili per operazioni di audit dei dati e per mantenere sincronizzati i dati disponibili.

Per creare un trigger a livello di riga si utilizza la clausola *FOR EACH ROW* nell'istruzione *CREATE TRIGGER*.

I trigger a livello d'istruzione vengono eseguiti **una sola volta** per ciascuna transazione, indipendentemente dal numero di righe modificate. Vengono utilizzati per attività correlate ai dati e per imporre misure aggiuntive di sicurezza sui tipi di transazione che possono essere eseguiti su una tabella.

Il trigger a livello d'istruzione è il **tipo predefinito** nel comando *CREATE TRIGGER*.

I trigger si basano sul paradigma ECA (Evento-Condizione-Azione) e presentano la seguente struttura:

- L'istruzione *CREATE TRIGGER*, seguita dal *NOME* assegnato al trigger.
- Il tipo del trigger (*BEFORE/AFTER*) e l'evento scatenante (*INSERT/UPDATE/DELETE*)
- La tabella su cui effettuare le azioni: *ON Tabella*
- La clausola opzionale per avere un trigger a livello di riga: [*FOR EACH ROW*]
- La condizione opzionale che si deve verificare per eseguire il trigger: [*WHEN Condizione*]
- Il codice che definisce l'azione da eseguire se si verifica la *Condizione*.

Ad esempio:

```
CREATE TRIGGER ControlloStipendio
BEFORE INSERT ON Impiegati
DECLARE
    StipendioMedio FLOAT
BEGIN
    SELECT avg(Stipendio) INTO StipendioMedio
    FROM Impiegati
    WHERE Dipartimento = :new.Dipartimento;
    IF :new.Stipendio > 2 * StipendioMedio
    THEN RAISE_APPLICATION_ERROR(-2061, 'Stipendio alto')
    END IF;
END;
```

Esistono due tipi di trigger, all'interno dei quali è possibile fare riferimento a vecchi e nuovi valori coinvolti nella transazione:

- **BEFORE**: il trigger viene eseguito **prima** dell'utilizzo delle istruzioni *INSERT*, *UPDATE* e *DELETE*.
Se si tratta di un trigger *BEFORE UPDATE*:
 - Per valori **vecchi** si intendono i valori che sono **nella tabella** e sono da modificare.
 - Per valori **nuovi** si intendono i valori **da inserire** al posto di quelli vecchi.
- **AFTER**: il trigger viene eseguito **dopo** dell'utilizzo delle istruzioni *INSERT*, *UPDATE* e *DELETE*.
Se si tratta di un trigger *AFTER UPDATE*:
 - Per valori **vecchi** si intendono i valori presenti **prima** della modifica.
 - Per valori **nuovi** si intendono i valori presenti nella tabella **alla fine** della modifica.

Inoltre, esiste il tipo **INSTEAD OF**, il quale permette di specificare **cosa fare invece di eseguire le azioni che hanno attivato il trigger**. I trigger di questo tipo devono essere a **livello di riga** e possono essere definiti su **viste**.

Ad esempio, è possibile reindirizzare una *INSERT* in una tabella verso una tabella differente; oppure, aggiornare con una *UPDATE* più tabella che fanno parte di una vista.

Un trigger può essere:

- **Attivo**: modifica lo stato della base di dati in corrispondenza di certi eventi.
- **Passivo**: provoca il fallimento di una transazione corrente in corrispondenza di certe condizioni.

Ogni componente/risorsa dello schema può essere **protetto**. Il possessore della risorsa, ovvero colui che la crea, **assegna dei privilegi** agli altri utenti. Un utente predefinito (`_system`) rappresenta l'**amministratore** della base di dati e ha il completo accesso alle risorse.

Ogni privilegio è caratterizzato da:

- La **risorsa** a cui si riferisce.
- L'utente che **concede** il privilegio.
- L'utente che **riceve** il privilegio.
- L'azione che viene **permessa** sulla risorsa.
- Se il privilegio può essere **trasmesso o meno** ad altri utenti.

I tipi di privilegio disponibili sono:

- **SELECT**: per la lettura dei dati.
- **INSERT [(Attributi)]**: per inserire righe con valori non nulli per gli attributi.
- **DELETE [(Attributi)]**: per cancellare delle righe.
- **UPDATE [(Attributi)]**: per modificare delle righe o degli attributi.
- **REFERENCES [(Attributi)]**: per definire chiavi esterne in altre tabelle che riferiscono gli attributi.

Solo il **creatore dello schema** della base di dati può eseguire le istruzioni **CREATE**, **ALTER** e **DROP**.

Il **creatore di una tabella** stabilisce i modi in cui altri possono farne uso mediante le istruzioni:

- **GRANT**: per concedere i privilegi a un utente.

GRANT Privilegi ON Risorsa TO Utenti [WITH GRANT OPTION]

Il comando **WITH GRANT OPTION** specifica se l'utente ha il privilegio di **trasferire i privilegi** ad altri utenti.

- **REVOKE**: per revocare i privilegi a un utente.

REVOKE Privilegi ON Risorsa FROM Utenti [RESTRICT | CASCADE]

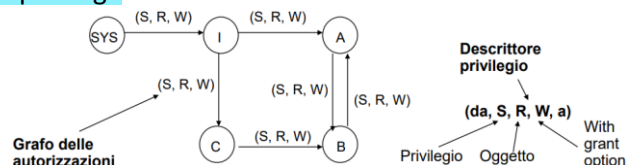
La revoca dev'essere fatta dall'utente che aveva concesso i privilegi, dove:

- **RESTRICT (default)**: specifica che il comando non dev'essere eseguito se la revoca dei privilegi all'utente comporta qualche altra revoca, dovuta a un precedente **GRANT OPTION**.
- **CASCADE**: forza l'esecuzione del comando.

È importante fare attenzione alle **reazioni a catena**, dato che togliendo i privilegi a un utente, li si tolgono anche a tutti gli utenti che li avevano ricevuti da quell'utente.

Il **creatore di una vista** possiede i privilegi che ha sulle tabelle usate nella definizione.

Il **grafo delle autorizzazioni** è un grafo con nodi etichettati collegati da archi diretti che indicano le relazioni tra chi concede e chi riceve i privilegi.



Se un nodo *N* ha un arco uscente con un privilegio, allora esiste un cammino da *SYSTEM* a *N* con ogni arco etichettato dallo stesso privilegio, più il comando **WITH GRANT OPTION (WGO)**.

Gli **indici** sono strutture dati che mappano i valori di una o più colonne di una tabella alle righe corrispondenti. Vengono utilizzati per ottimizzare le ricerche e migliorare le prestazioni di una base di dati. Non essendo un comando standard dell'SQL, presentano differenze tra i vari sistemi.

Per **creare un indice** si utilizza l'istruzione **CREATE INDEX Nome ON Tabella (Attributi)**

Per **eliminare un indice** si utilizza l'istruzione **DROP INDEX Nome**

Due metodi per produrre uno schema relazionale sono:

- Partire da un buono schema a oggetti e tradurlo.
- Partire da uno schema relazionale fatto da altri e modificarlo/completarlo.

La **teoria della progettazione relazionale** studia cosa sono le anomalie e come eliminarle (**normalizzazione**).

Possibili anomalie sono:

- Ridondanze.
- Potenziali inconsistenze.
- Anomalie negli inserimenti.
- Anomalie negli aggiornamenti.
- Anomalie nelle eliminazioni.

Lo **schema di relazione universale** di una base di dati relazionale ha come attributi l'unione degli attributi di tutte le relazioni della base di dati.

Una **forma normale** è una proprietà di una base di dati relazionale che ne garantisce la “**qualità**”, ovvero l'assenza di determinati difetti. Quando una relazione non è normalizzata:

- Presenta ridondanze.
- Si presta a comportamenti poco desiderabili durante gli aggiornamenti.

La **normalizzazione** è una procedura che permette di trasformare schemi non normalizzati in schemi che soddisfano una forma normale.

Di seguito, delle linee guida per una corretta progettazione relazionale:

- **Semantica degli attributi**: progettare ogni schema relazionale in modo che sia **semplice spiegarne il significato**. Non unire attributi provenienti da più tipi di classi, e tipi di associazione, in un'unica relazione.
- **Ridondanza**: progettare gli schemi relazionali in modo che nelle relazioni **non siano presenti anomalie** di inserimento, modifica o cancellazione. In presenza di anomalie da voler mantenere, è necessario assicurarsi che i programmi che aggiornano la base di dati operino correttamente.
- **Valori nulli**: evitare di porre in relazione di base attributi i cui valori possono essere **frequentemente nulli**. Se inevitabile, assicurarsi che si presentino in casi eccezionali e che non riguardino una maggioranza di tuple nella relazione.
- **Tuple spurie**: progettare schemi relazionali in modo tale che possano essere **riuniti tramite JOIN** con condizione di uguaglianza su attributi che sono **chiavi primarie o esterne**, in modo da garantire che non vengano generate **tuple spurie** (tuple contenenti valori non validi).
Inoltre, non avere relazioni contenenti attributi di “accoppiamento” diversi dalle combinazioni chiave esterna-chiave interna.

Per formalizzare la nozione di schema senza anomalie, è necessaria una descrizione formale della semantica dei fatti rappresentati in uno schema relazionale.

Un'**istanza valida di una relazione R** è una nozione semantica che dipende dalla conoscenza del dominio del discorso. È non estensionale e non deducibile da alcune istanze dello schema.

Informalmente

Sia r un'**istanza valida** su $R(T)$ e siano X e Y due **sottoinsiemi non vuoti** di T .

Allora, esiste in r una **dipendenza funzionale** da X a Y se, per ogni coppia di tuple t_1 e t_2 di r con gli stessi valori su X , risulta che t_1 e t_2 hanno gli stessi valori anche su Y .

La dipendenza funzionale da X a Y si denota con $X \rightarrow Y$

Esempio:

Persone (CodiceFiscale, Cognome, Nome, DataNascita)

CodiceFiscale $\rightarrow$ Cognome, Nome

Una dipendenza funzionale è un vincolo; perciò, non ci sono gradi di libertà.

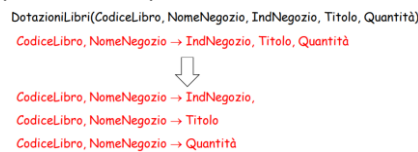
Formalmente

Dato uno schema $R(T)$ e $X, Y \subseteq T$, una **dependenza funzionale** tra gli attributi X e Y è un vincolo su R sulle istanze della relazione, espresso nella forma $X \rightarrow Y$

Perciò, **X determina funzionalmente Y o Y è determinato da X** , se per ogni istanza valida r di R , un valore di X determina in modo univoco un valore di Y ($\forall r$ istanza valida di $R, \forall t_1, t_2 \in r$ se $t_1[X] = t_2[X]$ allora $t_1[Y] = t_2[Y]$)

Un'istanza r_θ di R soddisfa la **dependenza funzionale** $X \rightarrow Y$ (ovvero $r_\theta \models X \rightarrow Y$) se la proprietà vale per r_θ . Inoltre, un'istanza r_θ di R soddisfa un insieme F di **dependenze funzionali** se, per ogni $X \rightarrow Y \in F$, vale $r_\theta \models X \rightarrow Y$

Le dipendenze funzionali del tipo $X \rightarrow Y$ si chiamano **dependenze funzionali atomiche**. Ogni dipendenza funzionale $X \rightarrow Y_1 Y_2 \dots Y_n$ si può scomporre in n dipendenze funzionali atomiche ($X \rightarrow Y_1, X \rightarrow Y_2, \dots, X \rightarrow Y_n$).



Una **dependenza funzionale banale** è del tipo *Impiegato, Progetto* $\rightarrow$ *Progetto*, ovvero **sempre valida**. Perciò, $X \rightarrow Y$ è **non banale** se Y non è contenuta in X (siamo interessati alle dipendenze funzionali non banali).

Si consideri la dipendenza funzionale *NomeNegozio* $\rightarrow$ *IndirizzoNegozio*.

Le dipendenze funzionali si possono esprimere, in modo equivalente:

- **Mediante un'espressione diretta** ($P \Rightarrow Q$): se in due righe il *NomeNegozio* è uguale, allora anche l'*IndirizzoNegozio* lo è ($\text{NomeNegozio}_= \Rightarrow \text{IndirizzoNegozio}_=$).
- **Per contrapposizione** ($\neg Q \Rightarrow \neg P$): se in due righe l'*IndirizzoNegozio* è diverso, allora anche il *NomeNegozio* lo è ($\text{IndirizzoNegozio}_\neq \Rightarrow \text{NomeNegozio}_\neq$).
- **Per assurdo**: non possono esserci due tuple con *NomeNegozio* uguale e *IndirizzoNegozio* diverso ($\text{NomeNegozio}_= \wedge \text{IndirizzoNegozio}_\neq \Rightarrow \text{False}$).

In generale, $A \Rightarrow B \Leftrightarrow \neg B \Rightarrow \neg A \Leftrightarrow A \wedge \neg B \Rightarrow \text{False}$

La notazione $R\langle T, F \rangle$ denota uno schema con attributi T e dipendenze funzionali F .

Le dipendenze funzionali sono una **proprietà semantica**, ovvero dipendono dai fatti rappresentati e non da come gli attributi sono combinati in schemi di relazione.

Una dipendenza funzionale è **completa** quando $X \rightarrow Y$ e, per ogni $W \subset X$, non vale $W \rightarrow Y$; inoltre:

- Se X è una **superchiave**, allora essa determina ogni altro attributo della relazione $X \rightarrow T$
- Se X è una **chiave**, allora $X \rightarrow T$ è una **dependenza funzionale completa**.

Da un insieme F di dipendenze logiche, in generale altre dipendenze logiche sono "implicate" da F .

- **Dipendenze implicate**: sia F un insieme di dipendenze funzionali sullo schema R . Si può dire che **F implica logicamente $X \rightarrow T$** ($F \models X \rightarrow Y$) se ogni istanza r di R che soddisfa F , soddisfa anche $X \rightarrow T$
- **Dipendenze banali**: sono implicate dal vuoto ($\{\} \models X \rightarrow X$).

Per derivare le dipendenze funzionali implicate logicamente da F si utilizza un insieme di **regole di inferenza**, ovvero gli **"assiomi" di Armstrong**:

- **Riflessività R**: se $X \subseteq Y$, allora $X \rightarrow Y$
- **Arricchimento A**: se $X \rightarrow Y$ e $Z \subseteq T$, allora $XZ \rightarrow YZ$
- **Transitività T**: se $X \rightarrow Y$ e $Y \rightarrow Z$, allora $X \rightarrow Z$

Sia F un insieme di dipendenze funzionali. Si può dire che $X \rightarrow Y$ è **derivabile** da F ($F \vdash X \rightarrow Y$) se $X \rightarrow Y$ può essere inferito da F utilizzando gli assiomi di Armstrong.

Una **derivazione** di f da F è una sequenza finita $f_1, \dots, f_m$ di dipendenze dove $f_m = f$ e ogni f_i è un elemento di F oppure è ottenuta dalle precedenti dipendenze $f_1, \dots, f_{i-1}$ della derivazione utilizzando una regola di inferenza.

È possibile dimostrare che valgono le seguenti regole:

- **Unione U:** $\{X \rightarrow Y, X \rightarrow Z\} \vdash X \rightarrow YZ$
- **Decomposizione D:** $Z \subseteq Y \{X \rightarrow Y\} \vdash X \rightarrow Z$

Teorema

Gli assiomi di Armstrong sono corretti e completi.

Dimostrazione

Mediante gli assiomi di Armstrong, è possibile mostrare l'equivalenza della nozione di **implicazione logica** ($\models$) e di quella di **derivazione** ($\vdash$).

Se una dipendenza è derivabile con gli assiomi di Armstrong, allora è anche implicata logicamente (**correttezza** degli assiomi); viceversa, se una dipendenza è implicata logicamente, allora è anche derivabile dagli assiomi (**completezza** degli assiomi).

- **Correttezza** degli assiomi: $\forall f, F \vdash f \Rightarrow F \models f$
- **Completezza** degli assiomi: $\forall f, F \models f \Rightarrow F \vdash f$

Dato un insieme F di dipendenze funzionali, la **chiusura** di F è $F^+ = \{X \rightarrow Y \mid F \vdash X \rightarrow Y\}$

Dato $R \langle T, F \rangle$ e $X \subseteq T$, la **chiusura di X rispetto a F** è $X_F^+ = \{A_i \in T \mid F \vdash X \rightarrow A_i\}$

Il **problema dell'implicazione** consiste nel decidere se una dipendenza funzionale $X \rightarrow Y$ appartiene a F^+ .

La sua risoluzione avviene mediante un algoritmo che genera F^+ applicando ripetutamente gli assiomi di Armstrong a F . Questo algoritmo ha una complessità esponenziale rispetto al numero di attributi dello schema e scaturisce dal teorema $F \vdash X \rightarrow Y \Leftrightarrow Y \subseteq X_F^+$

Sia X un insieme di attributi e F un insieme di dipendenze. L'algoritmo **calcola X_F^+** come segue:

1. Inizializza X^+ con l'insieme X
2. Se tra le dipendenze di F c'è una dipendenza $Y \rightarrow A$, con $Y \subseteq X^+$, allora inserisce A in X^+ ($X^+ = X^+ \cup \{A\}$)
3. Ripete il passo 2. fino a quando non ci sono altri attributi da aggiungere a X^+
4. Restituisce $X_F^+ = X^+$

L'algoritmo **termina**, dato che a ogni passo viene aggiunto un nuovo attributo a X^+ , e il numero di attributi è finito. Per dimostrarne la correttezza, è necessario dimostrare, per induzione, che $X_F^+ = X^+$.

Dato lo schema $R \langle T, F \rangle$, una **chiave candidata** di R è un insieme di attributi $W \subseteq T$ se:

- W è una **superchiave**, ovvero $W \rightarrow T \in F^+$
- W non è una **superchiave**, dove $V \subset W$, ovvero $\forall V \subset W, V \rightarrow T \notin F^+$

L'**attributo primo** è un attributo che appartiene ad almeno una chiave.

Il problema di trovare **tutte le chiavi di una relazione** richiede un algoritmo di complessità esponenziale, nel caso peggiore; mentre, il problema di **controllare se un attributo è primo** è NP-completo.

L'algoritmo per trovare tutte le chiavi si basa su due proprietà:

- Se un attributo A di T **non appare a destra** di alcuna dipendenza in F , allora A **appartiene** a ogni chiave di R .
- Se un attributo A di T **appare a destra** di alcuna dipendenza in F , ma **non appare a sinistra** di alcuna dipendenza non banale, allora A **non appartiene** ad alcuna chiave di R .

Due insiemi di dipendenze funzionali F e G , sullo schema R , sono **equivalenti** $F \equiv G \Leftrightarrow F^+ = G^+$

Se $F \equiv G$, allora F è una **copertura** di G , e viceversa.

Sia F un insieme di dipendenze funzionali:

- Data una dipendenza funzionale $X \rightarrow Y \in F$, si dice che X contiene un **attributo estraneo** A_i se e solo se $(X - \{A_i\}) \rightarrow Y \in F^+$, ovvero $F \vdash (X - \{A_i\}) \rightarrow Y$
Per stabilire se in una dipendenza funzionale del tipo $AX \rightarrow B$, l'attributo A è estraneo, è necessario calcolare X^+ e verificare se include B , ovvero se basta X a determinare B .

Esempio:

```
Orari(CodAula, NomeAula, Piano, Posti, Materia, CDL, Docente, Giorno, Ora)
• se vale
  • Docente, Giorno, Ora → CodAula
  • Docente, Giorno → Ora
• allora
  • Docente, Giorno → CodAula
  • (quindi) nella prima dipendenza Ora è attributo estraneo
```

- Una dipendenza funzionale $X \rightarrow Y$ si dice **dipendenza ridondante** se e solo se $(F - \{X \rightarrow Y\})^+ = F^+$, ovvero $F - \{X \rightarrow Y\} \vdash X \rightarrow Y$

Per stabilire se in una dipendenza funzionale del tipo $X \rightarrow A$ è ridondante, è necessario eliminarla da F , calcolare X^+ e verificare se include A , ovvero se con le dipendenze funzionali rimanenti è ancora possibile dimostrare che X determina B .

Esempio:

```
Orari(CodAula, NomeAula, Piano, Posti, Materia, CDL, Docente, Giorno, Ora)
• se vale
  • Docente, Giorno, Ora → CodAula
  • CodAula → NomeAula
• è inutile avere anche
  • Docente, Giorno, Ora → NomeAula
```

L'insieme di dipendenze funzionali F è detto una **copertura canonica** se e solo se:

- La parte destra di ogni dipendenza funzionale in F è un attributo.
- Non esistono attributi estranei.
- Nessuna dipendenza in F è ridondante.

Teorema

Per ogni insieme di dipendenze F esiste una copertura canonica.

L'algoritmo per calcolare una copertura canonica esegue i seguenti passi:

1. **Crea le dipendenze funzionali atomiche/Trasforma le dipendenze** nella forma $X \rightarrow A$: sostituisce l'insieme dato con quello equivalente avente tutti i secondi membri costituiti da singoli attributi (dipendenze atomiche).
2. **Elimina gli attributi estranei**: per ogni dipendenza verifica se esistono attributi eliminabili dal primo membro.
3. **Elimina le dipendenze ridondanti**.

Le **ridondanze sui dati** possono essere di due tipi:

- **Ridondanza concettuale**: non ci sono duplicazioni dello stesso dato, ma sono memorizzate informazioni che possono essere **ricavate da altre** contenute nella base di dati.
- **Ridondanza logica**: esistono duplicazioni sui dati che possono generare **anomalie nelle operazioni** sui dati.

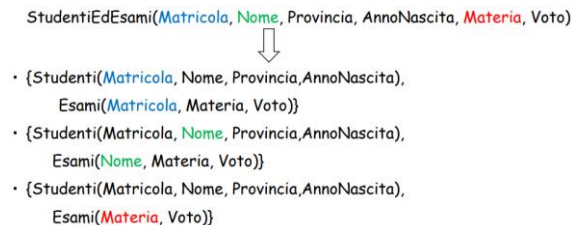
Di seguito, alcune **caratteristiche delle dipendenze funzionali**:

- Sono definite a **livello di schema**, non livello d'istanza.
- Hanno sempre un **verso**.
- Sono una **generalizzazione del vincolo di chiave** e di **superchiave**.

Perciò, data una tabella con schema $R(X)$, con superchiave K , esiste un vincolo di dipendenza funzionale tra K e qualsiasi attributo dello schema.

La **decomposizione di schemi** si utilizza per **eliminare anomalie** da uno schema, andandolo a decomporre in **schemi più piccoli "equivalenti"**. Si devono quindi "estrarre" gli attributi che sono determinati da attributi non chiave, ovvero "creare uno schema per ogni funzione"; non essendo così semplice, è necessario effettuare anche altre considerazioni.

Dato uno schema $R(T)$, $\rho = \{R_1(T_1), \dots, R_k(T_k)\}$ è una **decomposizione** di R se e solo se $T_1 \cup \dots \cup T_k = T$.
Ad esempio:



Due proprietà desiderabili di una decomposizione sono:

- **Conservazione dei dati:** $\rho = \{R_1(T_1), \dots, R_k(T_k)\}$ è una **decomposizione** di uno schema $R(T)$ che **preserva i dati** se e solo se, per ogni **istanza valida** r di R , $r = (\pi_{T_1} r) \wedge \dots \wedge (\pi_{T_k} r)$

Per non avere una perdita d'informazione, è necessario effettuare una decomposizione utilizzando la chiave primaria.

Uno schema $R(X)$ si decompone **senza perdita di dati** negli schemi $R_1(X_1)$ e $R_2(X_2)$ se, per ogni possibile istanza r di $R(X)$, il **join naturale** delle proiezioni di r su X_1 e X_2 produce la **tabella di partenza**, ovvero **non contiene tuple spurie** ($\pi_{X_1} r \bowtie \pi_{X_2} r = r$).

Teorema

Se l'insieme degli attributi comuni alle due relazioni ($X_1 \cap X_2$) è **chiave per almeno una** delle due relazioni decomposte, allora la decomposizione è **senza perdita**.

Sia r una relazione su un insieme di attributi X e siano X_1 e X_2 due sottoinsiemi di X la cui unione sia pari a X stesso; inoltre, sia X_0 l'intersezione di X_1 e X_2 . Allora r si decompone senza perdita su X_1 e X_2 se soddisfa la dipendenza funzionale $X_0 \rightarrow X_1$, oppure la dipendenza funzionale $X_0 \rightarrow X_2$.

Teorema decomposizione binaria

Sia $R\langle T, F \rangle$ uno schema di relazione. La decomposizione $\rho = \{R_1(T_1), R_2(T_2)\}$ **preserva i dati** se e solo se $T_1 \cap T_2 \rightarrow T_1 \in F^+$, oppure $T_1 \cap T_2 \rightarrow T_2 \in F^+$

Esistono criteri anche per decomposizioni in più di due schemi.

Anche se una decomposizione è senza perdite, può comunque presentare dei problemi di conservazione delle dipendenze.

- **Conservazione delle dipendenze:** una decomposizione **conserva le dipendenze** se ciascuna delle dipendenze funzionali dello schema originario coinvolge attributi che compaiono tutti insieme in uno degli schemi decomposti.

Dato lo schema $R\langle T, F \rangle$ e $T_1 \subseteq T$, la **proiezione di F su T_1** è $\pi_{T_1}(F) = \{X \rightarrow Y \in F^+ \mid XY \subseteq T_1\}$

Esempio

Sia $R(A, B, C)$ e $F = \{A \rightarrow B, B \rightarrow C, C \rightarrow A\}$

$\pi_{AB}(F) = \{A \rightarrow B, B \rightarrow A\}$

$\pi_{AC}(F) = \{A \rightarrow C, C \rightarrow A\}$

Dato lo schema $R\langle T, F \rangle$, la decomposizione $\rho = \{R_1, \dots, R_n\}$ **preserva le dipendenze** se e solo se l'unione delle dipendenze in $\pi_{T_i}(F)$ è una **copertura di F** . Questo problema ha complessità di tempo polinomiale.

Teorema

Sia $\rho = \{R_1, \dots, R_n\}$ una decomposizione di $R \langle T, F \rangle$ che preserva le dipendenze e tale che T_j , per qualche j , è una superchiave per $R \langle T, F \rangle$. Allora, ρ **preserva i dati**.

Una decomposizione dev'essere **senza perdita**, per garantire la ricostruzione delle informazioni originarie senza generazione di tuple spurie, e dovrebbe **preservare le dipendenze**, così da garantire il mantenimento dei vincoli d'integrità di dipendenza funzionale originari.

Una **forma normale** è una proprietà di una base di dati relazionale che ne garantisce la "qualità", ovvero l'assenza di determinati effetti. Quando una relazione **non è normalizzata**:

- Presenta **ridondanze**.
- Si presta a **comportamenti poco desiderabili** durante gli aggiornamenti.

Le forme normali sono:

- **1FN**: impone una restrizione sul **tipo di una relazione** (ogni attributo ha un tipo elementare).
- **2FN, 3FN e FNBC**: impongono restrizioni sulle **dipendenze**.

La **forma normale di Boyce-Codd (FNBC)** è la più naturale e la più restrittiva.

Una relazione r è in FNBC se, per ogni dipendenza funzionale non banale $X \rightarrow Y$ definita su di essa, X **contiene una chiave K di r** , ovvero una **superchiave**.

Teorema

Lo schema $R \langle T, F \rangle$ è in FNBC se e solo se, per ogni $X \rightarrow A \in F$ non banale, X è una superchiave.

La forma normale richiede che i concetti in una relazione siano omogenei, ovvero **solo proprietà direttamente associate alla chiave**.

Perciò, se esiste in R una dipendenza $X \rightarrow A$ non banale, e X **non è una chiave**, allora X modella l'identità di un'entità con attributi X^+ diversa da quelle modellate dall'intera R .

L'**algoritmo di normalizzazione/analisi** riceve in ingresso lo schema $R \langle T, F \rangle$ con copertura canonica F e restituisce una decomposizione in FNBC che **preserva i dati**; perciò, lo schema $R \langle T, F \rangle$ viene decomposto in $R_1(X, Y)$ e $R_2(X, Z)$, sui quali si ripete il procedimento.

Di seguito l'algoritmo:

```

 $\rho = \{R \langle T, F \rangle\}$ 
while esiste in  $\rho$  una  $R_i \langle T_i, F_i \rangle$  non in FNBC per la dipendenza funzionale  $X \rightarrow A$ 
do
     $T_a = X^+$ 
     $F_a = \pi_{T_a}(F_i)$ 
     $T_b = T_i - X^+ + X$ 
     $F_b = \pi_{T_b}(F_i)$ 
     $\rho = \rho - R_i + \{R_a \langle T_a, F_a \rangle, R_b \langle T_b, F_b \rangle\}$ 
end

```

Questo algoritmo preserva i dati, ma **non necessariamente le dipendenze**.

Non è sempre possibile ottenere una decomposizione in FNBC; quindi, occorre ricorrere a una forma normale indebolita.

La **terza forma normale (3FN)** è un target di normalizzazione che consente di ottenere automaticamente:

- Decomposizioni senza perdita.
- Decomposizioni che preservano tutte le dipendenze.

Una relazione r è in 3FN se, per ogni dipendenza funzionale non banale $X \rightarrow Y$ definita su r , è verificata **almeno una** delle seguenti condizioni:

- **X contiene una chiave K di r** , come nella FNBC.
- **Ogni attributo in Y è contenuto in almeno una chiave K di r .**

La terza forma normale è **meno restrittiva** della FNBC, dato che tollera alcune ridondanze e anomalie sui dati e certifica meno la qualità dello schema ottenuto. Però, qualsiasi sia lo schema di partenza, è **sempre ottenibile** mediante l'algoritmo di normalizzazione in TFN.

Teorema

Lo schema $R\langle T, F \rangle$ è in 3FN se, per ogni $X \rightarrow A \in F$ non banale, X è una superchiave oppure A è primo.

È possibile notare che la 3FN **ammette** una dipendenza non banale, e non da chiave, se gli attributi a destra sono primi. Invece, la FNBC **non ammette mai** nessuna dipendenza non banale e non da chiave.

Una versione base dell'**algoritmo di sintesi** riceve in ingresso un insieme R di attributi e un insieme F di dipendenze su R e restituisce una decomposizione $\rho = \{S_i\}_{i=1, \dots, n}$ di R tale che preservi dati e dipendenze e ogni S_i sia in 3FN rispetto alle proiezioni di F su S_i .

Di seguito i passi dell'algoritmo:

1. Trovare una copertura canonica G di F e porre $\rho = \{\}$
2. Sostituire in G ogni insieme $X \rightarrow A_1, \dots, X \rightarrow A_h$ di dipendenze con lo stesso determinante, con la dipendenza $X \rightarrow A_1 \cdot \dots \cdot A_h$
3. Per ogni dipendenza $X \rightarrow Y$ in G , mettere uno schema con attributi XY in ρ
4. Eliminare ogni schema di ρ contenuto in un altro schema di ρ
5. Se la decomposizione non contiene alcuno schema i cui attributi costituiscano una superchiave per R , aggiungere alla decomposizione lo schema con attributi W , dove W è una chiave di R .

Un **DBMS** è un sistema software che gestisce grandi quantità di dati persistenti e condivisi. La **gestione di grandi quantità di dati** richiede particolare attenzione ai **problemi di efficienza**. La **persistenza** e la **condivisione** richiedono che un DBMS fornisca dei meccanismi per garantire l'**affidabilità dei dati** (fault tolerance), per il **controllo degli accessi** e per il **controllo della concorrenza**.

Sono disponibili diverse altre funzionalità per motivi di **efficacia**, ovvero per semplificare la descrizione dei dati, lo sviluppo delle applicazioni, l'amministrazione di una base di dati, ecc.

La gestione integrata e la condivisione dei dati permettono di **evitare ripetizioni** (ridondanza dovuta a copie multiple dello stesso dato) e, di conseguenza, un inutile spreco di risorse (memoria); inoltre, la ridondanza può dare luogo a **problemi di inconsistenza delle copie** e, in ogni caso, comporta la necessità di propagare le modifiche, ovvero un **ulteriore spreco di risorse** (CPU e rete).

Dal punto di vista dell'utente, una base di dati è vista come una collezione di dati che modellano una certa porzione della realtà d'interesse.

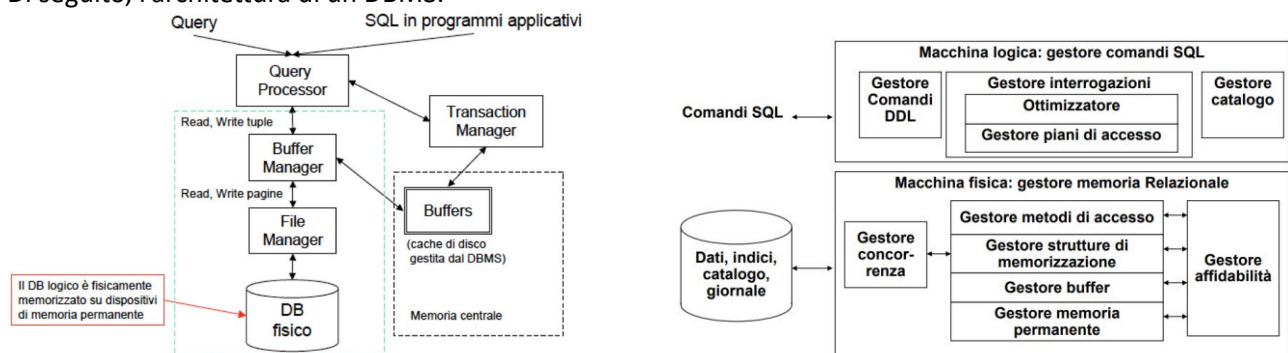
L'**astrazione logica** con cui i dati vengono resi disponibili all'utente definisce un **modello dei dati**, ovvero una collezione di concetti che vengono utilizzati per descrivere i dati, le loro relazioni e i vincoli che questi devono rispettare.

Un importante ruolo nella definizione di un modello dei dati è svolto dai meccanismi che possono essere utilizzati per strutturare i dati. Ad esempio, esistono modelli i cui dati sono descritti sotto forma di alberi (**modello gerarchico**), di grafi (**modello reticolare**), di oggetti complessi (**modello a oggetti**) e di relazioni (**modello relazionale**).

Un obiettivo di un DBMS è quello di fornire le caratteristiche di:

- **Indipendenza fisica**: l'organizzazione fisica dei dati dipende da considerazioni legate all'efficienza delle organizzazioni adottate. La riorganizzazione fisica dei dati **non deve comportare effetti collaterali** sui programmi applicativi.
- **Indipendenza logica**: permette di **accedere ai dati logici indipendentemente dalla loro rappresentazione fisica**.

Di seguito, l'architettura di un DBMS:



La memoria di un sistema di calcolo è organizzata in una **gerarchia**. Al livello più alto sono presenti memorie piccole e molto veloci, ma costose; scendendo lungo la gerarchia, la dimensione aumenta e diminuiscono velocità e costo.

Dato un indirizzo di memoria, le prestazioni si misurano in termini di **tempo d'accesso**, determinato dalla somma della **latenza** (tempo necessario per accedere al primo byte) e del **tempo di trasferimento** (tempo necessario per muovere i dati), ovvero
$$\text{Tempo d'accesso} = \text{latenza} + \frac{\text{dimensione dati da trasferire}}{\text{velocità di trasferimento}}$$

Una base di dati, a causa della sua dimensione, risiede normalmente su **dischi rigidi**. I dati, per essere elaborati dal DBMS, devono essere trasferiti in **memoria centrale**; questo trasferimento non avviene in termini di singole tuple, ma di **blocchi/pagine**.

Dato che le **operazioni di I/O** costituiscono il **collo di bottiglia** del sistema, è necessario ottimizzare l'implementazione fisica della base di dati attraverso:

- Un'organizzazione efficiente delle tuple sul disco.
- Delle strutture d'accesso efficienti.
- Una gestione efficiente dei buffer in memoria.
- Delle strategie di esecuzione efficienti per le query.

Un **hard disk** (HDD) è un dispositivo elettro-meccanico per la conservazione di informazioni, sotto forma magnetica, su un supporto rotante a forma di piatto su cui agiscono delle testine di lettura/scrittura.

La struttura della superficie di un piatto comprende:

- La **traccia**: organizzata in settori di dimensione fissa. I settori sono raggruppati logicamente in **blocchi**, ovvero l'**unità di trasferimento**. Trasferire un blocco richiede un tempo di posizionamento delle testine, un tempo di latenza rotazionale e un tempo per il trasferimento (trascurabile).
- Il **settore geometrico**.
- Il **settore di una traccia**.
- Il **cluster**, ovvero l'insieme di settori di tracce contigui.

Il meccanismo di un hard disk include organi di registrazione, di posizionamento e di rotazione. Un'**unità a dischi** contiene una pila di dischi metallici che ruota a velocità costante e alcune testine di lettura che si muovono radialmente al disco.

Un **blocco/pagina** è una sequenza contigua di settori su una traccia che costituisce l'unità di I/O per il trasferimento di dati da e per la memoria principale.

La dimensione tipica di una pagina è di qualche KB (da 4 a 64 KB):

- **Pagine piccole** comportano un maggior numero di operazioni di I/O.
- **Pagine grandi** tendono ad aumentare la **frammentazione interna**, ovvero pagine parzialmente riempite, e richiedono più spazio in memoria per essere caricate.

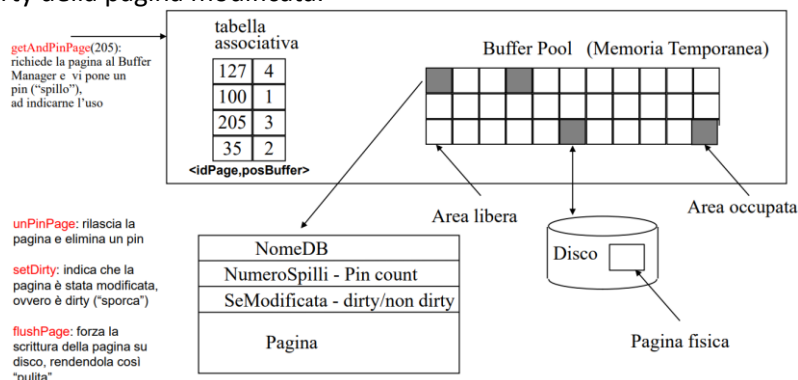
Il **tempo di trasferimento di una pagina** (T_t), da disco a memoria centrale, dipende dalla dimensione della pagina (P) e dal transfer rate (T_r). $T_t = P/T_r$

Il **gestore di memoria permanente** fornisce un'astrazione della memoria permanente in termini di insiemi di file logici di pagine fisiche di registrazioni, nascondendo le caratteristiche dei dischi e del sistema operativo.

Il **gestore del buffer** effettua il trasferimento delle pagine tra la memoria temporanea e la memoria permanente, offrendo agli altri livelli una visione della memoria permanente come un insieme di pagine utilizzabili in memoria temporanea, astraendo da quando vengono trasferite dalla memoria permanente al buffer, e viceversa.

Di seguito, l'**area delle pagine**, dove:

- **Pin count**: contatore che viene incrementato o decrementato quando una pagina viene richiesta o rilasciata.
- **Dirty/non dirty**: il componente che ha eseguito la modifica chiede al buffer di aggiornare la proprietà dirty/non dirty della pagina modificata.



Nei sistemi operativi, una politica comune adottata per decidere quale pagina rimpiazzare è la **Least Recently Used (LRU)**, ovvero viene rimpiazzata la pagina non utilizzata da più tempo.

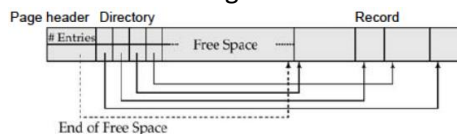
Nei DBMS, la LRU non è sempre una buona scelta, dato che per alcune query il "pattern d'accesso" ai dati è noto e può essere utilizzato per operare scelte più accurate, in grado di migliorare anche di molto le prestazioni.

L'**hit ratio**, ovvero la frazione di richieste che non provocano un'operazione di I/O, indica sinteticamente quanto buona è una politica di rimpiazzo.

Una pagina presenta:

- Una **struttura fisica**: un insieme di dimensione fissa di caratteri.
- Una **struttura logica**: informazioni di servizio e un'area contenente le stringhe che rappresentano le righe.

Il formato tipico di una pagina in un DBMS è come segue:



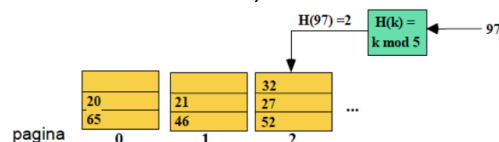
La **directory** contiene un puntatore per ogni riga della pagina; in questo modo, l'identificatore di una riga (**RID**) nella base di dati è formato da una coppia:

- **PID**: identificatore della pagina.
- **Slot**: posizione all'interno della pagina.

È possibile individuare velocemente una riga e permetterne la riallocazione nella pagina, senza modificare il RID.

I tipi di organizzazioni delle strutture di memorizzazione dei dati sono:

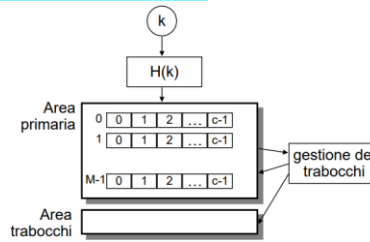
- **Seriali e sequenziali**:
 - **Organizzazione seriale (heap file)**: i dati sono memorizzati in modo **disordinato**, uno dopo l'altro. È l'organizzazione standard di ogni DBMS e le sue caratteristiche sono:
 - È **semplice** e con un **basso costo di memoria**.
 - È **poco efficiente**.
 - Va bene per **pochi dati**.
 - **Organizzazione sequenziale**: i dati sono **ordinati** sul valore di uno o più attributi. Le sue caratteristiche sono:
 - Effettua **ricerche più veloci**: utilizzando una ricerca binaria, richiede $\log_2 b_i + 1$ accessi alla pagina.
 - **Nuovi inserimenti fanno perdere l'ordinamento**.
- **Per chiave**: l'obiettivo è, noto il valore di una chiave, trovare la riga di una tabella con pochi accessi al disco (l'ideale sarebbe un unico accesso). I metodi per effettuare ciò sono:
 - Il **metodo procedurale (hash)**: in un file hash, le righe vengono allocate in una pagina il cui indirizzo **dipende dal valore di chiave** della riga: $key \rightarrow H(key) \rightarrow \text{indirizzo di pagina}$. Una funzione hash comune è il **resto della divisione intera** $H(k) = k \bmod NP$, la quale può essere applicata anche a chiavi alfanumeriche, una volta convertite.



I parametri di progetto sono:

- La **funzione per la trasformazione della chiave**.
- Il **fattore di caricamento** $d = N / (M * C)$, ovvero la frazione dello spazio fisico disponibile mediamente utilizzata. Se **N** è il **numero di tuple** previsto per il file, **M** il **fattore di pagine** e **C** il **fattore di caricamento**, il file può prevedere un **numero di blocchi B** pari al numero intero immediatamente superiore a **d**.

- La **capacità c** delle pagine.
- Il metodo per la **gestione dei trabocchi**.



Le collisioni (**overflow**) vengono gestite con le linked list.

L'**organizzazione statica** è la più efficiente per l'accesso diretto basato su valori della chiave con condizioni di uguaglianza (**accesso puntuale**); invece, non è efficiente per ricerche basate su intervalli. Inoltre, funziona solo con file la cui dimensione non varia molto nel tempo.

Gli **svantaggi** del metodo procedurale, ovvero l'utilizzo di strutture hash, sono:

- Dover effettuare dei **trade-off**:
 - Se il **numero dei blocchi è troppo piccolo** rispetto alla base di dati, si hanno **frequenti collisioni**.
 - Se il **numero di blocchi è troppo grande** rispetto alla base di dati, si ha un **fattore di riempimento dei blocchi molto basso**.
 - **Non è efficiente per le query su un intervallo di valori.**
 - **Non è efficiente per le operazioni che coinvolgono attributi che non sono chiave.**
- Il **metodo tabellare (indice)**: utilizza un **indice**, ovvero un **insieme ordinato di coppie $(k, r(k))$** , dove **k** è un **valore della chiave** e **$r(k)$** è un **riferimento alla riga con chiave k** .

L'indice è gestito con un'opportuna struttura ad albero, detta **B⁺-tree**; inoltre, gli indici possono essere **multi-attributi**.

L'**indice** è una struttura che contiene informazioni sulla posizione di memorizzazione delle tuple, sulla base del valore del campo **chiave**.

La **realizzazione degli indici** avviene attraverso l'utilizzo di strutture ad **albero multilivello**.

(Inizio ripasso)

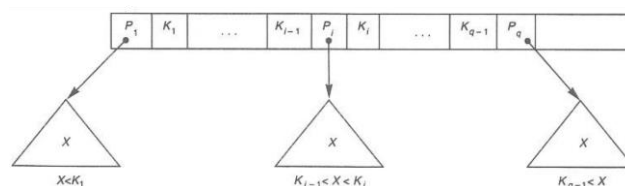
Un **albero binario di ricerca** è un albero binario etichettato, in cui, per ogni nodo:

- Il **sottoalbero sinistro** contiene solo etichette minori di quella del nodo.
- Il **sottoalbero destro** contiene solo etichette maggiori di quella del nodo.

Il tempo di ricerca e inserimento è pari alla profondità, ovvero logaritmico nel caso medio, assumendo un ordine di inserimento casuale.

Un **albero di ricerca di ordine p** è un albero i cui nodi contengono al massimo **$p - 1$** valori di ricerca e **p** puntatori, nel seguente ordine: **$\langle P_1, K_1, \dots, P_{q-1}, K_{q-1}, P_q \rangle$** , con **$q \leq p$**

Ogni **P_i** è un puntatore a un **nodo figlio** (o un puntatore nullo) e ogni **K_i** è un **valore di ricerca** appartenente a un insieme totalmente ordinato.



Ogni **albero di ricerca** deve soddisfare due vincoli fondamentali:

- In ogni nodo, si ha che **$K_1 < K_2 < \dots < K_{q-1}$**

- Per tutti i valori di X presenti nel sottoalbero puntato da P_i vale la seguente relazione:
 - $K_{i-1} < X$ per $i = q$
 - $K_{i-1} < X < K_i$, per $1 < i < q$
 - $X < K_i$, per $i = 1$

Un albero di ricerca può essere utilizzato per **cercare righe** memorizzate sul disco. Ogni ricerca o modifica comporta la visita di un cammino radice-foglia.

I valori di ricerca possono essere i valori di **uno dei campi del file**. A ogni valore di ricerca è associato un **puntatore alla riga** avente quel valore, oppure al **blocco** contenente quella riga, nel file di dati.

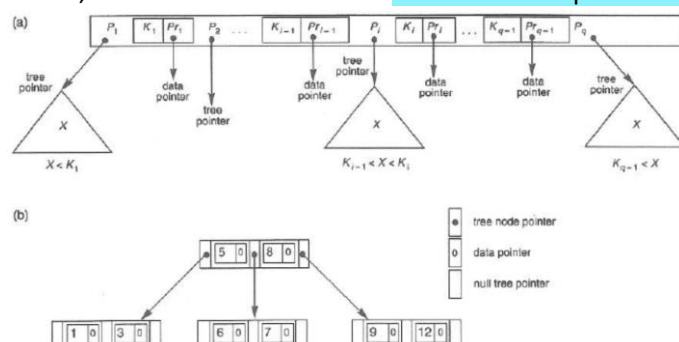
L'albero stesso può essere memorizzato sul disco, assegnando a ogni nodo dell'albero una **pagina**. Quando una nuova riga dev'essere inserita, l'albero di ricerca dev'essere **aggiornato** includendo il valore del campo di ricerca della nuova riga, con il **relativo puntatore** alla riga o alla pagina che lo contiene, nell'albero di ricerca.

Per inserire o cancellare valori di ricerca nell'albero di ricerca sono necessari specifici algoritmi che garantiscano il rispetto dei due vincoli fondamentali; però, tali algoritmi non garantiscono che un albero di ricerca risulti sempre **bilanciato**, ovvero che tutti i nodi foglia siano allo stesso livello.

(Fine ripasso)

Un **B-tree di ordine p** , se usato come struttura di accesso su un campo chiave per la ricerca di righe in un file di dati, deve soddisfare le seguenti condizioni:

- Ogni **nodo interno** del B-tree ha la forma:
 $\langle P_1, \langle K_1, Pr_1 \rangle, \dots, \langle P_{q-1}, \langle K_{q-1}, Pr_{q-1} \rangle, P_q \rangle$, con $q \leq p$, dove:
 - P_i è un puntatore a un altro nodo dell'albero (**tree pointer**).
 - K_i è la **chiave di ricerca**.
 - Pr_i è un puntatore a una riga il cui campo "chiave di ricerca" è uguale a K_i o alla pagina che contiene tale riga (**data pointer**).
- Per ogni nodo, si ha che $K_1 < K_2 < \dots < K_{q-1}$
- Ogni nodo ha, al **massimo, p tree pointer**.
- Per **tutti i valori X della chiave di ricerca** appartenenti al sottoalbero puntato da P_i , si ha che:
 - $K_{i-1} < X$ per $i = q$
 - $K_{i-1} < X < K_i$, per $1 < i < q$
 - $X < K_i$, per $i = 1$
- La **radice** ha almeno **due tree pointer**, a meno che non sia l'unico nodo dell'albero.
- Ogni nodo, esclusa la radice, ha almeno **$\lceil p/2 \rceil$ tree pointer**; inoltre, un nodo con q tree pointer, con $q \leq p$, ha **$q - 1$ campi "chiave di ricerca" e $q - 1$ data pointer**.
- **Tutti i nodi foglia sono posti allo stesso livello**. I nodi foglia hanno la stessa struttura dei nodi interni, a eccezione del fatto che **tutti i loro tree pointer P_i sono nulli**.



Un **B⁺-tree** è un B-tree in cui i data pointer sono memorizzati solo nei nodi foglia dell'albero. Dato che la struttura dei nodi foglia differisce dal B-tree, allora differisce anche quella dei nodi interni:

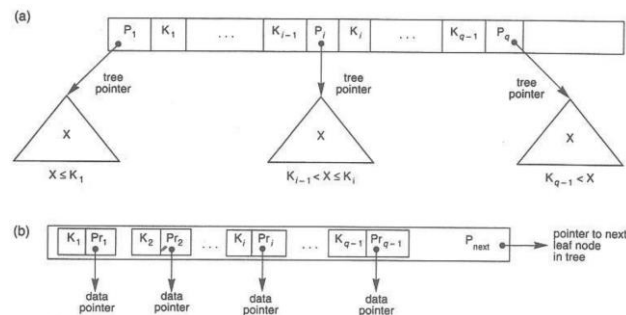
- Se il campo di ricerca è un campo chiave, allora i nodi foglia hanno, per ogni valore del campo di ricerca, una entry e un puntatore a una riga.
- Se il campo di ricerca non è un campo chiave, allora i puntatori indirizzano un blocco che contiene i puntatori alle righe del file di dati, rendendo così necessario un passo aggiuntivo per l'accesso ai dati.

Generalmente, i nodi foglia di un B⁺-tree sono messi in relazione tra di loro, così da essere sfruttato nel caso di range di query. I nodi foglia corrispondono al primo livello di un indice, mentre i nodi interni corrispondono agli altri livelli di un indice.

Inoltre, alcuni valori del campo di ricerca, presenti nei nodi foglia, sono ripetuti nei nodi interni per guidare la ricerca.

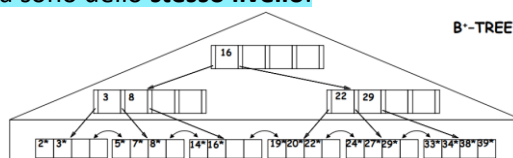
Di seguito, la struttura dei nodi interni di ordine p di un B⁺-tree:

- Ogni nodo interno del B⁺-tree ha la forma:
 $\langle P_1, K_1, \dots, P_{q-1}, K_{q-1}, P_q \rangle$, con $q \leq p$, dove P_i è un puntatore a un altro nodo del B⁺-tree (tree pointer).
- Per ogni nodo, si ha che $K_1 < K_2 < \dots < K_{q-1}$
- Ogni nodo interno ha, al massimo, p tree pointer.
- Per tutti i valori X della chiave di ricerca appartenenti al sottoalbero puntato da P_i , si ha che:
 - $K_{i-1} \leq X$ per $i = q$
 - $K_{i-1} < X \leq K_i$, per $1 < i < q$
 - $X < K_i$, per $i = 1$
- Ogni nodo interno, esclusa la radice, ha almeno $\lfloor p/2 \rfloor$ tree pointer
- Un nodo con q tree pointer, con $q \leq p$, ha $q - 1$ campi "chiave di ricerca".



Di seguito, la struttura dei nodi foglia di ordine p_{leaf} di un B⁺-tree:

- Ogni nodo foglia è della forma:
 $\langle \langle K_1, Pr_1 \rangle, \dots, \langle K_q, Pr_q \rangle, P_{next} \rangle$, con $q \leq p_{leaf}$, dove:
 - P_{next} è un tree pointer che punta al successivo nodo foglia del B⁺-tree.
 - Ogni Pr_i è un data pointer che punta alla riga con il valore "campo di ricerca" uguale a K_i o a un blocco contenente tale riga (oppure a un blocco di puntatori alle righe con il valore "campo di ricerca" uguale a K_i , se il campo di ricerca non è una chiave).
- Ogni nodo foglia ha almeno $\lfloor p_{leaf}/2 \rfloor$ valori.
- Tutti i nodi foglia sono dello stesso livello.



Solitamente, un **indice è organizzato a B⁺-tree** per permettere di trovare con pochi accessi, a partire da un valore v , le righe di R in cui il valore di A è in una **relazione specificata con v** .

Esistono due tipologie di indici ad albero:

- **Indici statici**: la struttura ad albero viene creata sulla base dei dati presenti nella base di dati e non viene più modificata (o viene modificata periodicamente).
- **Indici dinamici**: la struttura ad albero viene aggiornata a ogni operazione di inserimento o cancellazione sulla base di dati, preservando le prestazioni senza bisogno di riorganizzazioni.

Gli indici permettono di velocizzare l'accesso casuale, utilizzando la chiave di ricerca.

È possibile distinguere due tipi di indici:

- **Indice primario**: un file ordinato le cui righe sono di lunghezza fissa e sono costituite da due campi:
 - La **chiave primaria**, avente lo stesso tipo del campo chiave di ordinamento.
 - Il **puntatore a un blocco del disco**.

La chiave di ordinamento del file sequenziale **coincide** con la chiave di ricerca dell'indice.

- **Indice secondario**: definito su un campo non chiave che è una chiave candidata e ha valori unici, oppure su un campo non chiave con valori duplicati. Le righe sono costituite da due campi:
 - **Campo di indicizzazione**, avente lo stesso tipo del campo non utilizzato per ordinare il file.
 - **Puntatore al blocco/alla riga (RID)**.

La chiave di ordinamento e la chiave di ricerca sono **diverse**.

Un indice può essere definito su un **insieme $A_1, \dots, A_n$ di attributi**; in questo caso, l'indice contiene una riga **per ogni combinazione di valori** assunti dagli attributi $A_1, \dots, A_n$ nella relazione e può essere utilizzato per rispondere in modo efficiente a interrogazioni che specifichino un valore per **ciascuno di questi attributi**.

- **Per attributi non chiave**.

Mentre, i parametri che caratterizzano un'organizzazione sono:

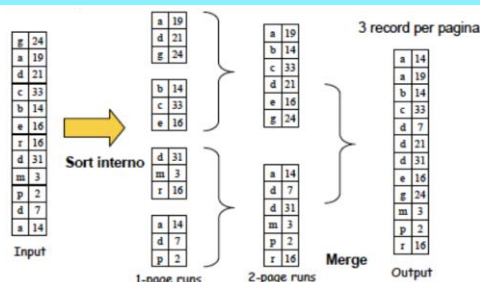
- Occupazione di memoria.
- Costo delle operazioni di ricerca per valore o intervallo, inserimento, modifica e cancellazione.

Nelle basi di dati è importante avere un ordinamento degli archivi, in modo da avere un risultato delle interrogazioni ordinato (*ORDER BY*) e per eseguire alcune operazioni relazionali (*JOIN*, *GROUP BY*, ...).

L'algoritmo più utilizzato dai DBMS è il **merge sort a Z vie (Z-way merge sort)**, il quale opera in due fasi:

1. **Sort interno**: legge, una alla volta, le pagine del file e ordina le righe di ogni pagina mediante l'uso di un **algoritmo di sort interno**. Ogni pagina ordinata, detta **run**, viene scritta sul disco in un file temporaneo.
2. **Merge**: mediante uno o più passi di fusione, le run vengono **fuse** fino a produrre un'unica **run**.

Ad esempio, nel **caso base di $Z = 2$** e un buffer in memoria centrale di dimensione $NB = 3$:



Considerando solo il numero di operazioni di I/O, il **costo complessivo è $2 * NP * (\lceil \log_2 NP \rceil + 1)$** , dove:

- **NP è il numero di pagine nel file.**
- Nella fase di sort interno, vengono lette e riscritte NP pagine.
- A ogni passo di merge, vengono lette e riscritte NP pagine.
- Il numero di passi di fusione è pari a $\lceil \log_2 NP \rceil$, dato che a ogni passo il numero di run si dimezza.

Nel **caso generale**, è possibile notare che il passo di sort interno, avendo a disposizione un buffer di dimensione NB , può ordinare NP pagine alla volta, invece che una sola. Questo porta il costo complessivo a $2 * NP * (\lceil \log_2(NP/NB) \rceil + 1)$.

Un miglioramento sostanziale si ottiene se, avendo a disposizione un **buffer di dimensione $NB > 3$** , si fondono $NB - 1$ run alla volta (un buffer viene utilizzato per l'output). Questo porta il costo complessivo a $2 * NP * (\lceil \log_{NB-1}(NP/NB) \rceil + 1)$.

Oltre all'ordinamento delle tuple, il **sort può essere utilizzato per:**

- Query in cui compare l'opzione ***DISTINCT***, ovvero per **eliminare i duplicati**.
- Query che contengono la clausola ***GROUP BY***.

Gli operatori relazionali realizzabili sono:

- **Proiezione:**

```
SELECT DISTINCT Provincia
FROM Studenti R
```

Mediante l'opzione ***DISTINCT*** viene utilizzato un approccio basato sull'ordinamento:

- Viene letto R e scritto T , il quale contiene solo gli attributi della ***SELECT***.
- Viene ordinato T su tutti gli attributi.
- Vengono eliminati i duplicati.

- **Selezione:**

```
SELECT *
FROM Studenti R
WHERE R.Provincia = "PI"
```

Dove:

- Senza indice e dati disordinati: $N \text{ pag}(R)$
- Con indice (B⁺-tree): $CI + CD$

- **Raggruppamento:**

- **Senza *GROUP BY***: vengono visitati i dati e calcolate le funzioni di aggregazione.

```
SELECT COUNT(*)
FROM Persone
```
- **Con *GROUP BY***: approccio basato sull'ordinamento che ordina i dati sugli attributi del ***GROUP BY***, visita i dati e calcola le funzioni di aggregazione per ogni gruppo.

```
SELECT Cognome, COUNT(Cognome)
FROM Persone
GROUP BY Cognome
```

- **Join:**

```
SELECT *
FROM Studenti S, Esami E
WHERE S.Matricola = E.Matricola
```

X

Nei **nested loops**, il costo d'esecuzione dipende dallo spazio a disposizione in memoria centrale.

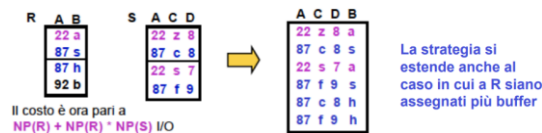
Nel **caso base**, con un buffer per R e uno per S , è necessario leggere una volta R e $N \text{ rec}(R)$ volte S , ovvero tante volte quante sono le **tuple della relazione esterna**, per un totale di $N \text{ pag}(R) + N \text{ rec}(R) * N \text{ pag}(S)$ operazioni di I/O.

Se è possibile allocare un **buffer di dimensione $N \text{ pag}(S)$** per la relazione interna, il costo si riduce a $N \text{ pag}(R) + N \text{ pag}(S)$

La relazione con **meno righe/pagine**, per motivi d'efficienza, dev'essere la **relazione esterna**.

L'ordine con cui vengono generate le tuple del risultato **coincide** con l'ordine eventualmente presente nella relazione esterna; perciò, se l'ordine che si genera è "interessante", la scelta della relazione esterna può risultarne **influenzata**.

Molti DBMS utilizzano una variante del nested loop, ovvero il **nested loop a pagine** che, rinunciando a **preservare l'ordine della relazione esterna**, risulta **più efficiente**, dato che esegue il join di tutte le tuple in memoria prima di richiedere nuove pagine della relazione interna.



Di seguito, l'algoritmo:

```
foreach r in R do
  foreach s in S where r.i = s.j do
    aggiunge <r, s> al risultato
```

Il **nested loop con indice**, data una tupla della relazione esterna R , effettua una scansione basata su un indice costruito sugli attributi di join di S , invece di effettuare una scansione completa della relazione interna S , secondo il seguente schema:



Di seguito, l'algoritmo:

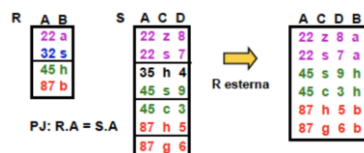
```
foreach r in R do
  foreach s in get-through-index(ISj=r.i) do
    aggiunge <r, s> al risultato
```

L'accesso alla relazione interna mediante l'indice, porta a ridurre i costi d'esecuzione.

Il **sort-merge join** è applicabile quando entrambi gli insiemi di tuple in input sono ordinati sugli attributi di join. L'algoritmo sfrutta il fatto che entrambi gli input sono **ordinati** per evitare di fare **confronti inutili**; in questo modo, il numero di lettura è dell'ordine di $N \text{ pag}(R) + N \text{ pag}(S)$ se accede **sequenzialmente** alle due relazioni.

Per $R(S)$ ciò è possibile se:

- $R(S)$ è fisicamente ordinata sugli attributi di join.
- Esiste un indice sugli attributi di join di $R(S)$



Di seguito, l'algoritmo:

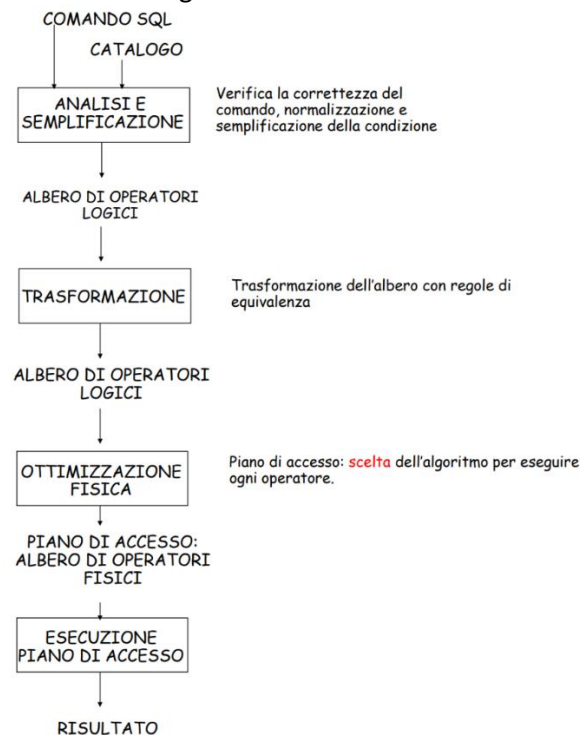
```
r = first(R)
s = first(S)
while r in R and s in S do
  if r.i = s.j
    avanza r e s fino a che r.i e s.j non cambiano entrambe
    aggiunge ciascun <r, s> al risultato
  else if r.i < s.j
    avanza r dentro R
  else if r.i > s.j
    avanza s dentro S
```


Perciò:

- Il **nested loop a pagine**, per ogni pagina di R , visita le pagine di S e trova le righe $\langle r, s \rangle$ del join, con r in R -pagina e s in S -pagina.
- Il **nested loop con indice** si utilizza quando esiste l'indice IS_j sull'attributo di join j della relazione interna S .
- Il **sort-merge** si utilizza quando R e S sono ordinate sull'attributo di join; esso visita R e S in parallelo.

I **piani d'accesso** descrivono il modo in cui la base di dati accederà ai dati. Il DBMS sceglie il piano con **costo minimo** tra i possibili piani alternativi, utilizzando le statistiche presenti nel catalogo.

Di seguito, le fasi del processo di ottimizzazione delle interrogazioni:



Gli operatori relazionali (proiezione, selezione, raggruppamento e join) possono essere realizzati con algoritmi diversi e codificati in opportuni **operatori fisici**.

Ogni **operatore fisico** è un **iteratore**, ovvero un oggetto con i seguenti metodi:

- **open**: inizializza lo stato dell'operatore, alloca il buffer per gli input e l'output e si richiama ricorsivamente sugli operatori figli. Viene utilizzato per **passare argomenti**, ad esempio la condizione che un operatore *Filter* deve applicare.
- **next**: viene utilizzato per richiedere un'altra tupla del risultato dell'operatore; perciò, **ritorna una riga**. L'implementazione di questo metodo include il metodo *next* sugli operatori figli e un codice specifico dell'operatore.
- **isDone**: indica se **sono ancora presenti dei valori da leggere**
- **close**: viene utilizzato per **terminare l'esecuzione dell'operatore**, con conseguente rilascio delle risorse ad esso allocate.
- **reset**

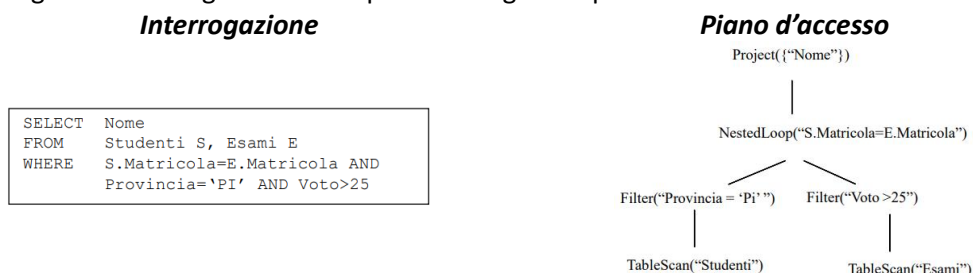
Questi metodi sono realizzati utilizzando gli operatori della macchina fisica.

Un **piano d'accesso** è un algoritmo per eseguire un'interrogazione utilizzando gli operatori fisici disponibili. Le **foglie** di un piano d'accesso sono le tabelle, mentre i **nodi interni** specificano le **modalità con cui gli accessi alle tabelle e le operazioni relazionali sono effettuate**.

Di seguito, la **tabella degli operatori logici e fisici**:

R	$TableScan(R)$: scansione di R
	$IndexScan(R, IDx)$: scansione di R con l'indice IDx
	$SortScan(R, \{A_i\})$: scansione di R ordinata sugli $\{A_i\}$
$\pi_{\{A_i\}}$ PROIEZIONE	$Project(O, \{A_i\})$: proiezione delle righe di O senza l'eliminazione dei duplicati
	$Distinct(O)$: eliminazione dei duplicati delle righe ordinate di O
σ_{Ψ} SELEZIONE	$Filter(O, \Psi)$: restrizione senza indici delle righe di O
	$IndexFilter(R, IDx, \Psi)$: restrizione con indice delle righe di R
$\tau_{\{A_i\}}$	$Sort(O, \{A_i\})$: ordinamento delle righe di O sugli $\{A_i\}$ per i valori crescenti
$\{A_i\} \gamma \{f_i\}$	$GroupBy(O, \{A_i\}, \{f_i\})$: raggruppamento delle righe di O sugli $\{A_i\}$ utilizzando le funzioni di aggregazione in $\{f_i\}$, dove: • L'insieme $\{f_i\}$ contiene le funzioni di aggregazione presenti nella <i>SELECT</i> e nella <i>HAVING</i> • L'operatore restituisce le righe con gli attributi $\{A_i\}$ e le funzioni in $\{f_i\}$ • Le righe di O sono ordinate sugli $\{A_i\}$
$\bowtie_{\Psi_j}$	$NestedLoop(O_E, O_I, \Psi_j)$: giunzione con il nested loop, con Ψ_j condizione di giunzione
	$PageNestedLoop(O_E, O_I, \Psi_j)$: giunzione con il page nested loop
	$IndexNestedLoop(O_E, O_I, \Psi_j)$: giunzione con l'index nested loop
	$SortMerge(O_E, O_I, \Psi_j)$: giunzione con il sort merge, con le righe di O_E e O_I ordinati sugli attributi di giunzione

Ad esempio, la seguente interrogazione corrisponde al seguente piano d'accesso:



Le **transazioni** rappresentano l'unità di lavoro elementare, ovvero l'insieme di istruzioni SQL che modificano il contenuto di una base di dati. Sintatticamente, una transazione è contornata dai comandi *begin transaction* ed *end transaction*, al quale interno possono comparire i comandi di *commit work* e *rollback work*.

Le proprietà **ACID** delle transazioni sono:

- **Atomicità**: la transazione dev'essere eseguita con la regola del "tutto o niente".

- **Consistenza**: la transazione deve lasciare la base di dati in uno stato consistente; perciò, eventuali vincoli di integrità non devono essere violati.
- **Isolamento**: l'esecuzione di una transazione deve essere indipendente dalle altre.
- **Persistenza** (Durability): l'effetto di una transazione che ha eseguito l'istruzione *commit work* non deve essere perso.

In breve, le funzioni del DBMS sono:

- La **gestione dei dati**: cura la memorizzazione permanente dei dati e il loro accesso.
- La **gestione del buffer**: cura il trasferimento dei dati da memoria di massa a memoria centrale e il caching dei dati in memoria centrale.
- L'**ottimizzazione delle interrogazioni**: seleziona il piano d'accesso di costo ottimo con cui valutare ogni interrogazione.

Una **transazione** è un'unità logica di elaborazione che corrisponde a una serie di operazioni fisiche elementari (letture e scritture) sulla base di dati.

Ad esempio, il trasferimento di una somma da un conto corrente all'altro:

```
UPDATE CC
SET Saldo = Saldo - 50
WHERE Conto = "123"
```

Una funzionalità essenziale di un DBMS è la **protezione dei dati** da malfunzionamenti e interferenze dovute all'**accesso contemporaneo** ai dati da parte di più utenti.

Per un programmatore, una **transazione** è un programma sequenziale costituito da operazioni che il sistema deve eseguire garantendo le proprietà ACID. Un'altra definizione di transazione è: una sequenza di azioni di lettura e scrittura in memoria permanente e di elaborazioni di dati in memoria temporanea.

Il **gestore dell'affidabilità** garantisce le seguenti proprietà:

- **Atomicità**: la transazione che terminano prematuramente (**aborted transactions**) sono trattate dal sistema come se non fossero mai iniziate; perciò, eventuali loro effetti sulla base di dati sono annullati.
- **Persistenza** (Durability): le modifiche sulla base di dati di una transazione terminata normalmente sono permanenti, ovvero non sono alterabili da eventuali malfunzionamenti.

Il **gestore della concorrenza** garantisce le seguenti proprietà:

- **Serializzabilità**: nel caso di esecuzioni concorrenti di più transazioni, l'effetto complessivo è quello di un'esecuzione seriale.

Per aumentare l'efficienza prestazionale, tutti i DBMS utilizzano un **buffer temporaneo** di informazioni in memoria principale, che viene periodicamente scritto su memoria secondaria.

Una transazione può eseguire molte operazioni sui dati recuperati da una base di dati, ma al DBMS interessano solo quelle di lettura o scrittura della base di dati, indicate con $r_i[x]$ e $w_i[x]$.

- Un'**operazione di lettura $r_i[x]$** comporta la lettura di una pagina del buffer, **se non già presente**.
- Un'**operazione di scrittura $w_i[x]$** comporta l'eventuale lettura e modifica di una pagina nel buffer, **ma non necessariamente** la sua scrittura in memoria permanente; per questo, in caso di malfunzionamento, si può perdere l'effetto dell'operazione.

Un dato letto o scritto può essere una riga, un campo di una riga o una pagina. Per semplicità si suppone sia una **pagina**.

I tipi di malfunzionamento sono:

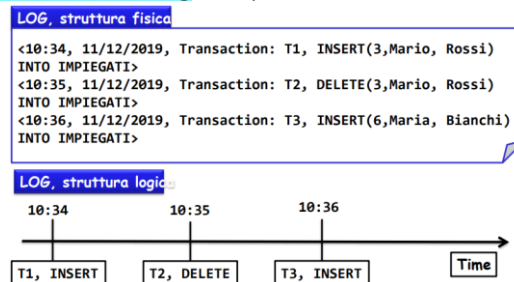
- I **fallimenti di transazioni**: **non** comportano la perdita di dati in memoria temporanea o persistente. Ad esempio, violazioni di vincoli e protezioni, stallo, ecc.

- I **fallimenti di sistema**: comportano la **perdita di dati in memoria temporanea**, ma non di dati in **memoria persistente**. Ad esempio, un comportamento anomalo del sistema, caduta di corrente, guasti hardware sulla memoria, ecc.
- I **disastri**: comportano la **perdita di dati in memoria persistente**. Ad esempio, il danneggiamento di periferica.

Il **gestore dell'affidabilità** verifica che siano garantite le proprietà di atomicità e persistenza delle transazioni. È responsabile di:

- Implementare i comandi di *begin transaction*, *commit* e *rollback*
- Ripristinare il sistema dopo malfunzionamenti software (**ripresa a caldo**).
- Ripristinare il sistema dopo malfunzionamenti hardware (**ripresa a freddo**).

Il **controllore di affidabilità** utilizza un **file di log** nel quale indicare tutte le operazioni svolte dal DBMS.



Le convenzioni notazionali sono, data una transazione T , un oggetto O , un before state BS e un after state AS

- $B(T)$ i record di *begin*
- $C(T)$ i record di *commit*
- $A(T)$ i record di *abort*
- $I(T, O, AS)$ i record di *INSERT*
- $U(T, O, BS, AS)$ i record di *UPDATE*
- $D(T, O, BS)$ i record di *DELETE*

I record del log associati a una transazione consentono di **disfare** e **rifare** le corrispondenti azioni sulla base di dati:

- La primitiva **undo**: per **disfare** un'azione su un oggetto O è sufficiente ricopiare in O il valore di BS ; l'inserimento viene disfatto cancellando l'oggetto O .
- La primitiva **redo**: per **rifare** un'azione su un oggetto O è sufficiente ricopiare in O il valore di AS ; la cancellazione viene rifatta cancellando l'oggetto O .

Il log si presenta come un **file sequenziale** suddiviso in record.



Esistono due tipi di record:

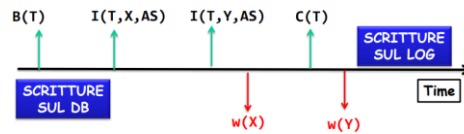
- **Record di transazione**: tengono traccia delle operazioni svolte da ciascuna transazione sul DBMS.
- **Record di sistema**: tengono traccia delle operazioni di sistema, come **dump** e **checkpoint**.

L'operazione di **dump** produce una copia completa della base di dati, in **mutua esclusione** con tutte le altre transazioni, quando il sistema non è operativo. La copia viene memorizzata in memoria stabile (**backup**). Al termine del dump, nel log viene scritto un **record di dump**, il quale segnala l'avvenuta esecuzione dell'operazione in un dato istante. Terminato ciò, il sistema riprende il suo funzionamento normale.

Il **log** è un file contenente la storia delle azioni effettuate sulla base di dati dal momento in cui ne è stata fatta l'ultima copia.

Per **scrivere sul log** è necessario seguire le seguenti regole:

- **Write Ahead Log (WAL):** la parte del before state BS di ogni record di log dev'essere scritta **prima** che la corrispondente operazione venga effettuata nella base di dati.
- **Commit Precedence:** la parte dell'after state AS di ogni record di log dev'essere scritta **prima** di effettuare il *commit* della transazione.



Gli algoritmi si differenziano a seconda del modo in cui si trattano le scritture sulla base di dati e la terminazione di transazioni:

- Disfare-Rifare
- Disfare-NonRifare
- NonDisfare-Rifare
- NonDisfare-NonRifare

Secondo la politica della **modifica libera**, le modifiche possono essere portate nella base di dati stabile prima che la transazione termini (disfare o *steal*).

La regola per poter **disfare una transazione (Write Ahead Log)** dice che, se la nuova versione di una pagina rimpiazza la vecchia sulla base di dati stabile, prima che la transazione abbia raggiunto il punto di *commit*, allora la vecchia versione della pagina dev'essere portata prima sul log in modo permanente.

Secondo la politica del **commit libero**, una transazione può essere considerata terminata normalmente anche prima che tutte le modifiche vengano riportate nella base di dati stabile (rifare).

La regola per poter **rifare una transazione (Commit Rule)** dice che, le modifiche di nuove versioni delle pagine di una transazione devono essere portate stabilmente nel log prima che la transazione raggiunga il *commit*.

Al momento del ripristino, solo gli **aggiornamenti più recenti**, tra quelli **riportati sul log**, potrebbero non essere stati ancora riportati sulla base di dati. Per ottenere la certezza che non sia necessario rieseguire le operazioni più vecchie, periodicamente viene eseguito un **checkpoint (CKP)**.

Sul log viene scritta la marca CKP per indicare che tutte le operazioni che la precedono sono state effettivamente effettuate sulla base di dati.

I passi da seguire per effettuare un checkpoint sono:

1. Sospendere l'attivazione di nuove transazioni.
2. Scrivere nel log la marca di **inizio checkpoint** che riporta le transazioni attive ($BeginCKP, \{T_1, \dots, T_n\}$).
3. Completare le transazioni precedenti per allineare la base di dati.
4. Scrivere nel log la marca di **fine checkpoint** $EndCKP$, la quale certifica che tutte le scritture avvenute prima dell'inizio del checkpoint sono sul disco.
5. Riprendere l'esecuzione delle operazioni.

Le scritture tra $BeginCKP$ e $EndCKP$ **non sono garantite essere sul disco**.

A seconda del tipo di malfunzionamento, il DBMS riprende in un certo modo:

- Per i **fallimenti di transazioni**: scrive nel log $(T, abort)$ e applica la procedura di disfare.
- I **fallimenti di sistema**: la base di dati viene ripristinata con il comando *restart*, a partire dall'ultimo **checkpoint**, procedendo come segue:
 - Le transazioni **non terminate** vengono **disfatte**.
 - Le transazioni **terminate** vengono **rifatte**.
- I **disastri**: riporta in linea la copia più recente della base di dati e la aggiorna rieffettuando le modifiche delle transazioni terminate normalmente (**ripartenza a freddo**).

La **ripresa a caldo** garantisce l'atomicità e la persistenza delle transazioni. È composta da quattro fasi:

- Trovare l'**ultimo checkpoint**, ripercorrendo il log a ritroso.
- Costruire gli insiemi **undo** (transazioni da disfare) e **redo** (transazioni da rifare).
- Ripercorrere il log all'**indietro**, fino alla più vecchia azione delle transazioni in undo, **disfacendo** tutte le azioni delle transazioni in **undo**.
- Ripercorrere il log in **avanti**, **rifacendo** tutte le azioni delle transazioni in **redo**.

La **ripresa a freddo** risponde a un guasto che provoca il deterioramento della base di dati. È composta da tre fasi:

- Ripristinare i dati a partire da un backup.
- Eseguire le operazioni registrate sul log fino all'istante del guasto.
- Eseguire una ripresa a caldo.

Uno **schedule S** si dice **seriale** se le azioni di ciascuna transazione appaiono in sequenza, senza essere inframmezzate da azioni di altre transazioni.

Uno **schedule seriale $S = \{T_1, \dots, T_n\}$** è ottenibile se:

- Le transazioni sono eseguite una alla volta (scenario non realistico)
- Le transazioni sono completamente indipendenti l'una dall'altra (scenario improbabile)

Il DBMS transazionale gestisce questi problemi garantendo la **proprietà d'isolamento**, la quale garantisce che una transazione sia eseguita come se non ci fosse concorrenza. Questa proprietà è assicurata facendo in modo che ciascun insieme di transazioni concorrenti sottoposte al sistema sia **serializzabile**.

In un sistema reale, per ragioni di efficienza e scalabilità, le transazioni vengono eseguite **in concorrenza**. È però necessario che il DBMS garantisca che l'esecuzione concorrente di transazioni avvenga senza interferenze, in caso di accessi agli stessi dati. Alcune interferenze sono:

- Una perdita di aggiornamento.
- Una lettura sporca/impropria.
- Una lettura inconsistente/non riproducibile.

Un'esecuzione di un insieme di transazioni $\{T_1, \dots, T_n\}$ si dice **seriale** se, per ogni coppia di transazioni T_i e T_j , tutte le operazioni di T_i vengono eseguite prima di qualsiasi operazione T_j , o viceversa.

Un'esecuzione di un insieme di transazioni si dice **serializzabile** se produce lo stesso effetto sulla base di dati di quello ottenibile eseguendo serialmente, in qualunque ordine, le sole transazioni terminate normalmente.

I DBMS implementano **tecniche di controllo della concorrenza**, le quali garantiscono direttamente la serializzabilità delle transazioni concorrenti. Queste tecniche si dividono in due classi:

- **Protocolli pessimistici/conservativi**: tendono a "ritardare" l'esecuzione di transazioni che potrebbero generare conflitti, e di conseguenza anomalie, rispetto alla transazione corrente; perciò, **cercano di prevenire**. Questo avviene mediante tecniche che si dividono in due classi principali:
 - **Metodi basati su lock**: per poter effettuare qualsiasi operazione di lettura/scrittura su una risorsa, è necessario aver precedentemente acquisito il controllo (**lock**) sulla risorsa stessa.

Il **lock** è un meccanismo che blocca l'accesso ai dati ai quali una transazione accede ad altre transazioni. Un lock può essere:

- A livello di riga, tabella o pagina (**multi-granularità**)
- In operazioni di **scrittura (mutua esclusione)** o di **lettura (accesso condiviso) (multimodale)**

In generale, quando una risorsa è **bloccata**, le transazioni che ne richiedono l'accesso vengono **messe in coda**; perciò, devono aspettare che il lock **venga rimosso**.

Questo meccanismo è efficace, ma **influisce sulle prestazioni**.

Su ogni lock possono essere definite due operazioni:

- **Richiesta** del lock in lettura/scrittura.
- **Rilascio** del lock (**unlock**) acquisito in precedenza.

Il gestore della concorrenza (**serializzatore**) dei DBMS ha il compito di stabilire l'ordine secondo il quale vanno eseguite le singole operazioni per rendere serializzabile l'esecuzione di un insieme di transazioni.

Il protocollo **Strict Two Phase Locking (2PL)** è definito dalle seguenti regole:

- Ogni transazione, **prima di effettuare un'operazione**, acquisisce il blocco corrispondente (ne **richiede il lock**).
- **Transazioni diverse** non ottengono blocchi in conflitto.
- **I lock vengono rilasciati alla terminazione della transazione**, ovvero al commit.

Perciò, **i lock di una transazione sono rilasciati solo dopo aver effettuato le operazioni di commit/abort**.

Il problema del 2PL è la possibile generazione di schedule con situazioni di stallo (**deadlock**). Per **gestire le situazioni di deadlock** causate dal gestore della concorrenza, è possibile usare tre tecniche:

- **Uso dei timeout**: ogni operazione di una transazione ha un **timeout** entro il quale dev'essere completata, **pena l'annullamento (abort)** della transazione stessa.
- **Deadlock avoidance**: per prevenire configurazioni che potrebbero portare a un deadlock, il **lock/unlock delle risorse dev'essere fatto allo stesso tempo**.
Per fare ciò, è possibile utilizzare il **timestamp** o le **classi di priorità tra transazioni**; questo **può portare alla starvation**, ovvero la situazione in cui una transazione è **impossibilitata a proseguire** la sua esecuzione per un periodo di **tempo indefinito**, mentre le altre transazioni del sistema **proseguono tranquillamente**.
- **Deadlock detection**: utilizzare **algoritmi per identificare** eventuali situazioni di deadlock e prevedere meccanismi di recovery dal deadlock.
Il **grafo delle richieste/risorse** può essere utilizzato per identificare la presenza di deadlock, ovvero di **cicli**; perciò, in caso di cicli, è necessario eseguire un'abort delle transazioni coinvolte nel ciclo, così da **eliminare la mutua dipendenza**.

- **Metodi basati su timestamp**: a ogni transazione viene associato un **timestamp** che rappresenta il momento di inizio della transazione:
 - Ogni transazione **non può leggere o scrivere** un dato **scritto** da una transazione con **timestamp maggiore**.
 - Ogni transazione **non può scrivere** su un dato **già letto** da una transazione con **timestamp maggiore**.
- **Protocolli ottimistici**: permettono l'esecuzione sovrapposta e non sincronizzata di transazioni ed **effettuano un controllo sui possibili conflitti generati solo a valle del commit**.

Ogni transazione effettua le **proprie operazioni** sugli oggetti della base di dati, secondo l'ordine temporale con cui le operazioni sono generate.

Al momento del commit, viene effettuato un controllo per stabilire se sono stati riscontrati **eventuali conflitti** e, nel caso, viene effettuato un **rollback** delle azioni delle transazioni e la relativa **riesecuzione**.

In generale, un protocollo di controllo di concorrenza ottimistico è basato su tre fasi:

1. **Fase di lettura**: ogni transazione legge i valori degli oggetti della base di dati su cui deve operare e li memorizza in variabili/copie locali, dove sono effettuati eventuali aggiornamenti.

2. **Fase di validazione**: vengono effettuati dei controlli sulla serializzabilità, nel caso in cui gli aggiornamenti locali delle transazioni dovessero essere propagati sulla base di dati.
3. **Fase di scrittura**: gli aggiornamenti delle transazioni che hanno superato la fase di validazione sono propagati definitivamente sugli oggetti della base di dati.

Di seguito, i **livelli di isolamento/consistenza** per ogni transazione:

- **SERIALIZABLE** assicura che:
 - Una transazione legge solo i cambiamenti fatti da **transazioni concluse**, ovvero che hanno fatto il commit.
 - Nessun valore letto o scritto da una transazione verrà cambiato da altre transazioni fino a che la transazione **non è conclusa**.
 - Se una transazione legge un insieme di valori acceduti secondo qualche condizione di ricerca, l'insieme **non viene modificato** da altre transazioni fino a che la transazione non è conclusa.
- **REPEATABLE READ** assicura che:
 - Una transazione legge solo i cambiamenti fatti da **transazioni concluse**, ovvero che hanno fatto il commit.
 - Nessun valore letto o scritto da una transazione verrà cambiato da altre transazioni fino a che la transazione **non è conclusa**.
- **READ COMMITTED** assicura che:
 - Una transazione legge solo i cambiamenti fatti da **transazioni concluse**, ovvero che hanno fatto il commit.
 - Una transazione, tra i valori letti all'inizio, non vede **nessun cambiamento effettuato** da transazioni concorrenti **non concluse**.
- **READ UNCOMMITTED** assicura che:
 - Una transazione può leggere modifiche fatte ad un oggetto da una **transazione in esecuzione**. L'oggetto **può essere cambiato** mentre la transazione è in esecuzione; perciò, la transazione è soggetta a **effetti fantasma**.